



Fundamentals of Database System

Chapter 3: Entity Relationship Model

Compiled by: Diriba G. (MSc. in CSE)

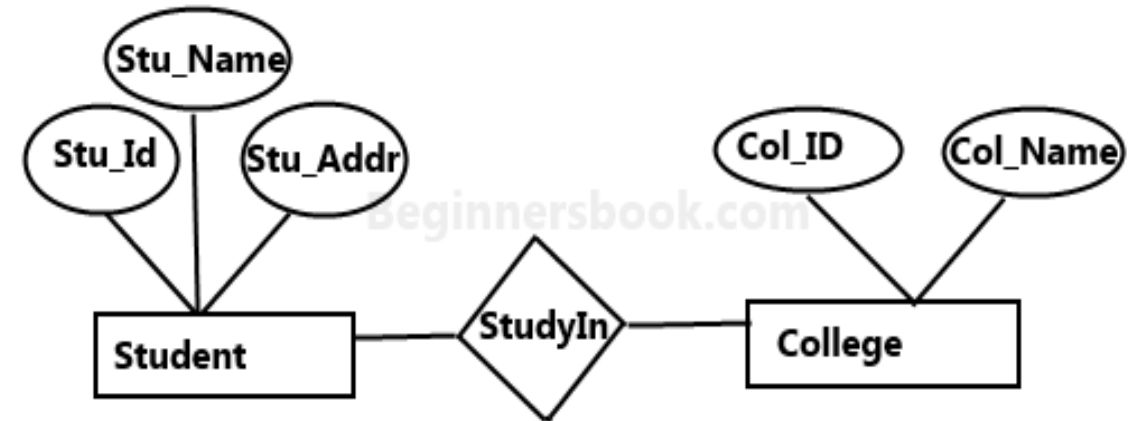
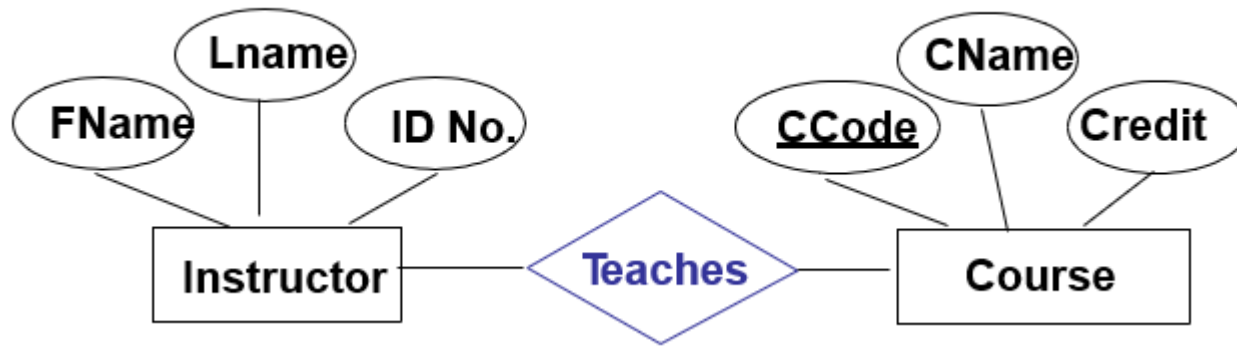
Outline

- ❖ Introduction to ER model
- ❖ ER Model Components
 - ❖ Entity
 - ❖ Attributes
 - ❖ Relationships
- ❖ ER diagram Notations
- ❖ The ER Diagram
- ❖ Mapping ER models to relational tables

ER Model: Introduction

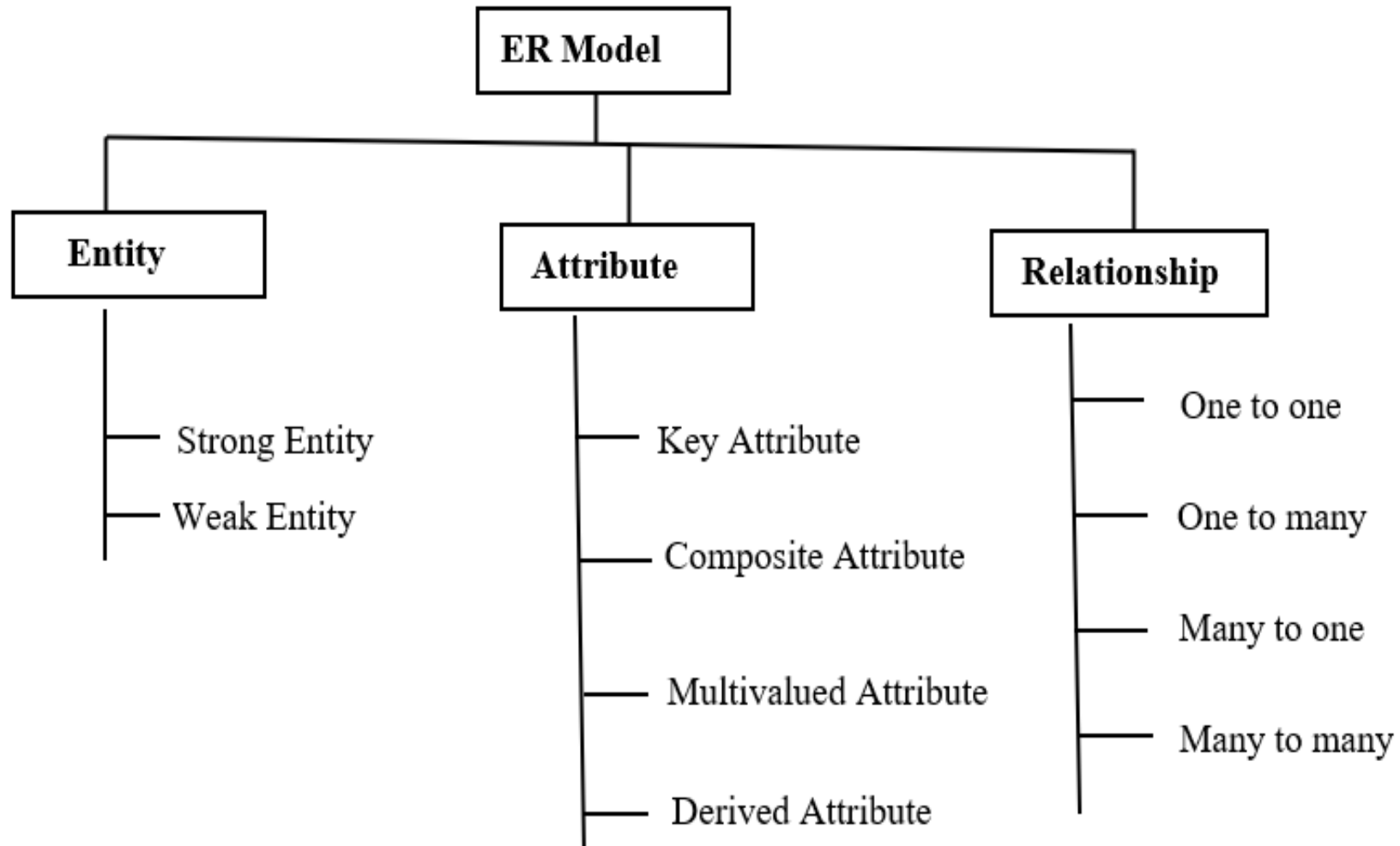
- ❖ ER Model is a high level or conceptual model
- ❖ The ER model is used to define the data elements and relationships for a specified system.
- ❖ It develops conceptual design for the database. It also develops a very simple and easy to design view of data.
- ❖ In ER modeling, the database structure is portrayed as a diagram called ER diagram.

Simple ER model examples



What did you understand from the above ER models?

Components of ER model



Entity

- ❖ **Entities** are specific objects or things in the mini-world that are represented in the database.
- ❖ An entity may be person, place or object.
- ❖ In ER diagram entity is represented as rectangles.

Example:



- ❖ Employee and Department are entities and represented as rectangle

Cont..

- ❖ An **entity type** defines a collection of entities that have the same attributes.
- ❖ An **entity instance** is a single item in this collection.
- ❖ An **entity set** is a set of entity instances.
- ❖ Examples:
 - ✓ Entity type: STUDENT
 - ✓ Entity instance: Student with ID number CS08
 - ✓ Entity set: Collection of all students
- ❖ In a database, an entity set is represented by the Table.

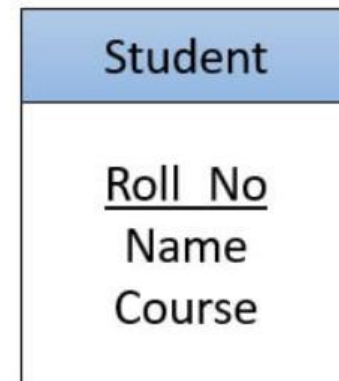
Example: Student table (Entity set)
i.e. each students in the table represents
Entity instance (just entity)

Roll No.	Name	Course
CS08	Steive	Comp. Sci.
EE54	Jhoson	Electronics
B12	Eva	Biology
F32	Jhoson	Finance
M26	Erica	Maths

Types of entity set

- ❖ **Strong Entity:** An entity set that has a **primary key** using which, entities in the table can be uniquely identified.
- ❖ Strong entity set is also known as a **regular entity set**.
- ❖ In an ER diagram,
 - ✓ The strong entity set is represented by the **rectangle and**
 - ✓ The primary key is underlined with the **solid line**.

Example: Student is **strong entity set** having primary key.

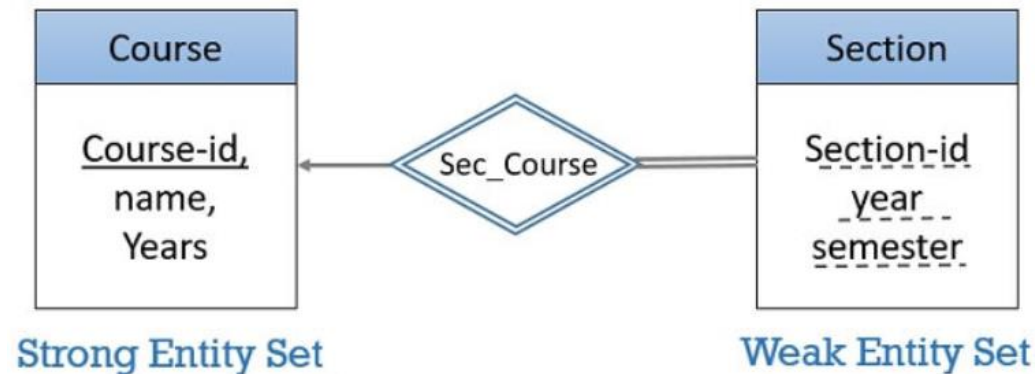


Cont..

- ❖ **Weak entity:** A weak entity set **doesn't have any primary key** which can identify each entity in a set distinctly.
- ❖ But, for discriminating the entities in a set, the weak entity set is **dependent** on a particular **strong entity set**.
- ❖ The relation between a weak entity set and a **strong entity** set is said to be **identifying relationship**.
- ❖ The weak entity is shown by a **rectangle** in an ER diagram. But, the attributes of weak entity set are underlined with the **dashed line**.

Cont..

- ❖ A weak entity is also said to be **existence dependent** as for the existence of its entities it has to be dependent on **identifying entity set** i.e. a particular ‘strong entity set’.
- ❖ The relationship of a weak entity set and strong entity set is shown with the **double diamond** as shown in the figure below:



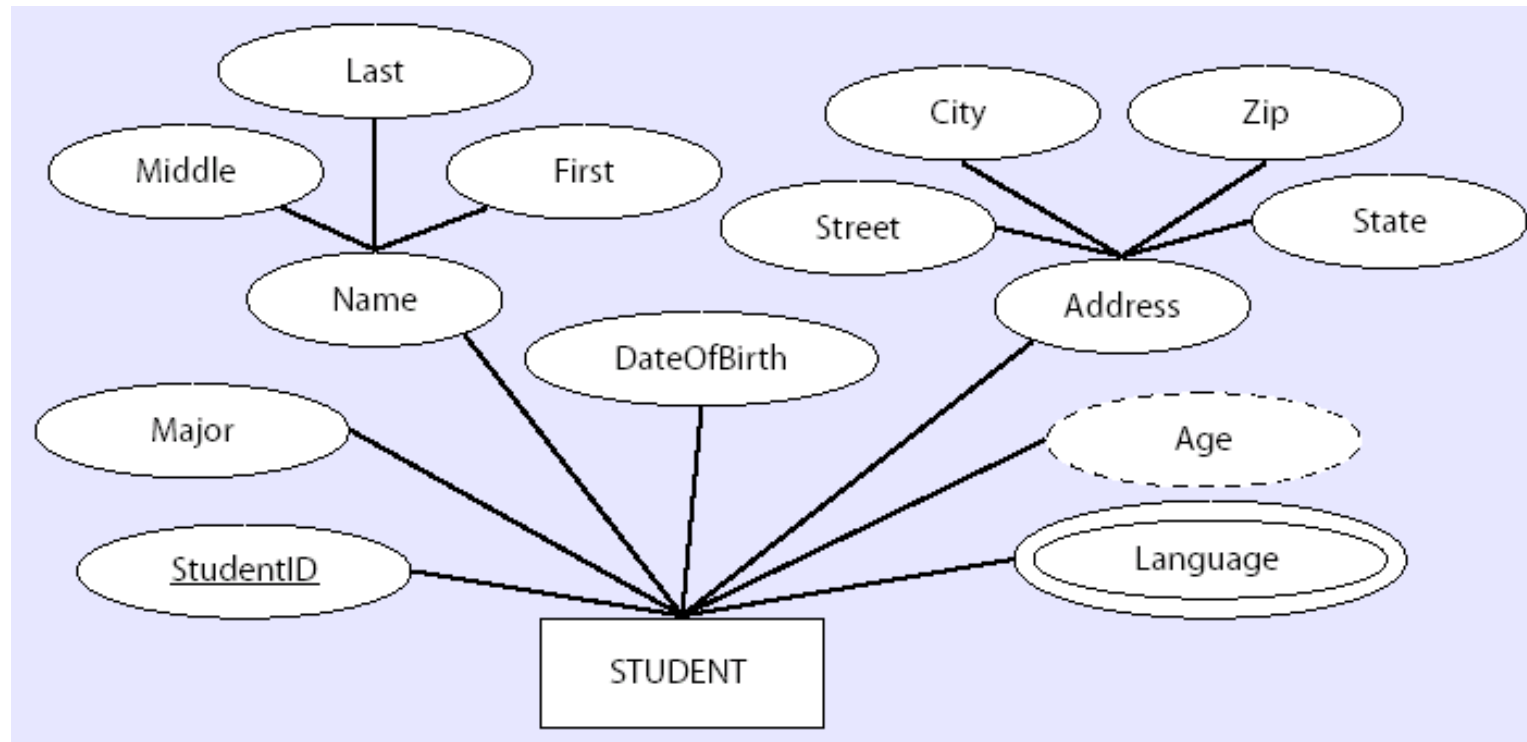
Attributes

- ❖ An **attribute** is a property or characteristic of an entity type.
- ❖ We describe an entity with a set of attributes
- ❖ Examples:
 - STUDENT = {Reg. No, Fname , LName, Address, Phone, Email, DOB}
 - ORDER = {Order ID, Date of Order, Amount of Order}
 - ACCOUNT = {Account No, Account type, Balance}
 - CITY = {Name, State, Population}

Types of Attributes

❖ Attributes can be simple, multivalued, composite, stored, or derived attributes.

❖ **Example:** Student attributes in ER diagram.



Cont..

- ❖ A set of attributes describe an entity.
- ❖ A particular **instance** of an attribute is called a **value**.
- ❖ For example, “Employee Id” and “Name” are the attributes of the “EMPLOYEES” entity set; and “Jones” is one **value** of the attribute “Name”.
- ❖ The **domain** of an attribute is the collection of all possible values an attribute can have. The domain of “Name” is a character string

Cont..

- ❖ Attributes can be classified as **identifiers** or **descriptors**.
- ❖ **Identifiers**: more commonly called keys, uniquely identify an instance of an entity.
- ❖ Example: “Employee Id” uniquely identifies an employee entity from the entity set.
- ❖ **Descriptor**: describes a non-unique characteristic of an entity instance.
- ❖ Example: “Name” is a descriptor for the “EMPLOYEES” entity set.

Simple vs composite attributes

❖ A **simple** or an **atomic attribute** cannot be further divided into smaller components.

❖ Example:

✓ Age

✓ Gender

❖ A **composite attribute**, however, can be divided into smaller subparts where each subpart is either atomic or composite.

❖ Examples:

- Name: First Name, Last Name
- Address: Street, City, State, ZipCode

Single valued vs multi valued attributes

❖ **Single-valued attributes** have a single value for an entity instance.

❖ **Examples:**

- Date Of Birth
- Reg. No

❖ **Multi-valued attributes**, on the other hand, may have more than one value for an entity instance.

❖ In ER model, it is represented by doubled ellipse.

❖ **Examples:**

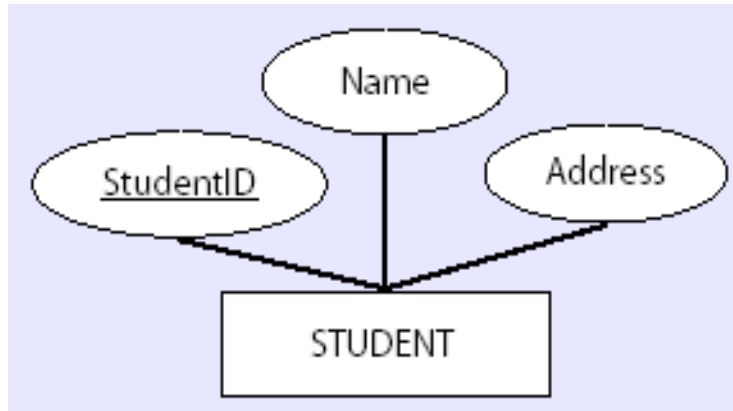
- Languages: Stores the names of the languages that a student speaks
- College Degree: BSC., MSC. ,M.Tech. ,PHD, etc.
- Phone number: Mobile phone , Office phone , Home phone

Stored vs derived attributes

- ❖ The value of a **derived attribute** can be determined by analyzing other attributes
- ❖ It is denoted with a dashed ellipse
- ❖ Example:
 - ✓ Age: Can derived from the current date and the attribute Date_of_Birth
- ❖ An attribute whose value cannot be derived from the values of other attributes is called a **stored attribute**.
- ❖ **NULL Values** the meaning of the former type of NULL is *not applicable* , NULL can also be used if we do not know the value of an attribute for a particular entity

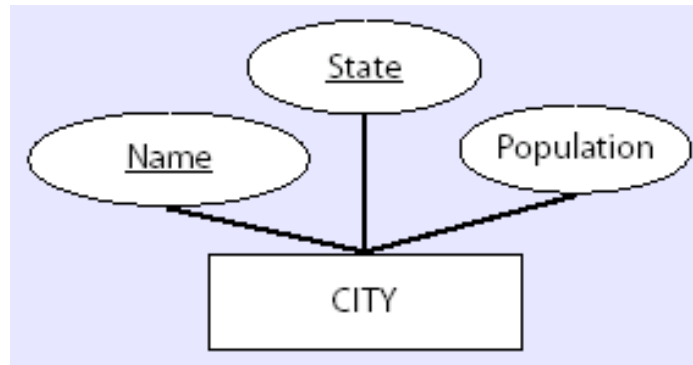
Key attributes

- ❖ A **key attribute** (or identifier) is a single attribute or a combination of attributes that uniquely identify an individual instance of an entity type.
- ❖ Underlined in an E-R diagram.
- ❖ Example: studentID is a key attribute uniquely identifying the student.



Candidate key attribute

- ❖ Sometimes no single attribute can uniquely identify an instance of an entity type
- ❖ A **composite key** is a composite attribute that uniquely identifies each entity instance.
- ❖ Example: composite key attribute
 - ✓ City: {Name, State}

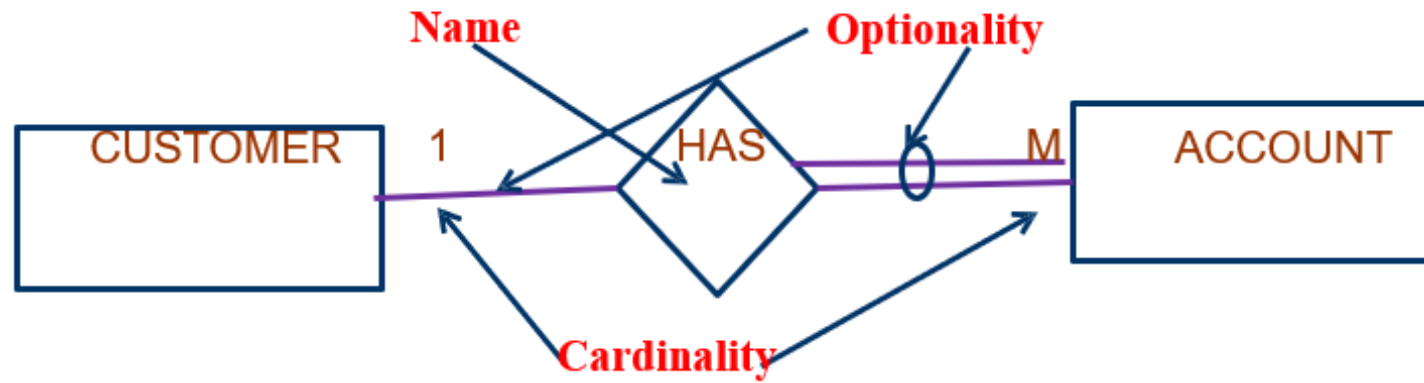


Relationships

- ❖ A **relationship** is an association, a linkage or a connection between or among entities.
- ❖ Example:
 - INSTRUCTOR “teaches” COURSE
 - STUDENT “takes” COURSES
 - CUSTOMER “has” ACCOUNT
- ❖ A relationships of the same type are grouped into **relationship type**
- ❖ A relationship type is represented by **diamond** connected to relating entities by lines.
- ❖ A relationship should have a **name** , an **optionality** (optional or mandatory) and a **cardinality**.

Cont..

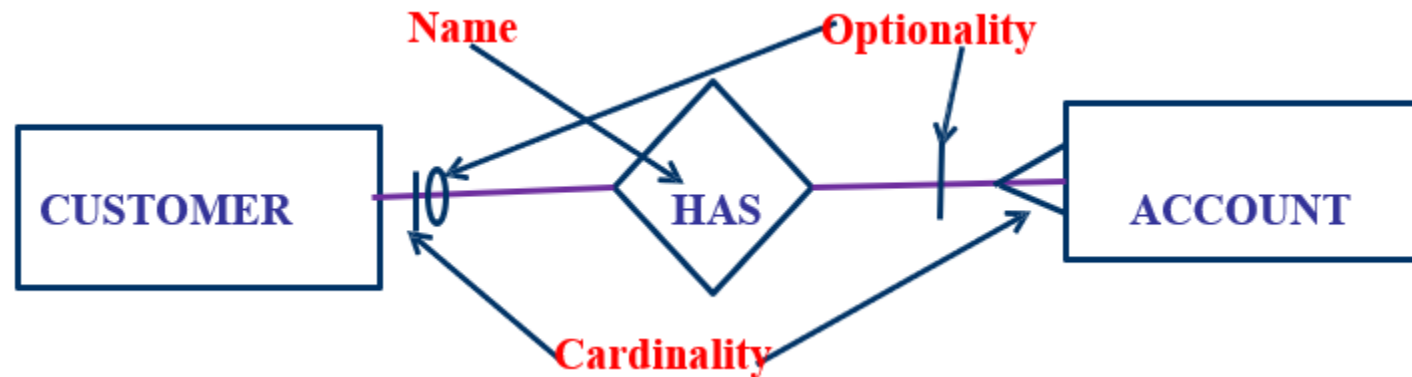
❖ Example: CUSTOMER “has” ACCOUNT



❖ Relationship between CUSTOMER and ACCOUNT entities

Cont..

- ❖ There are different notations for E-R diagrams.
- ❖ **The notation used in the previous slide is called Chen's notation.**
- ❖ **Other notations are also used. The notation shows below is called Crow's-feet notation.**



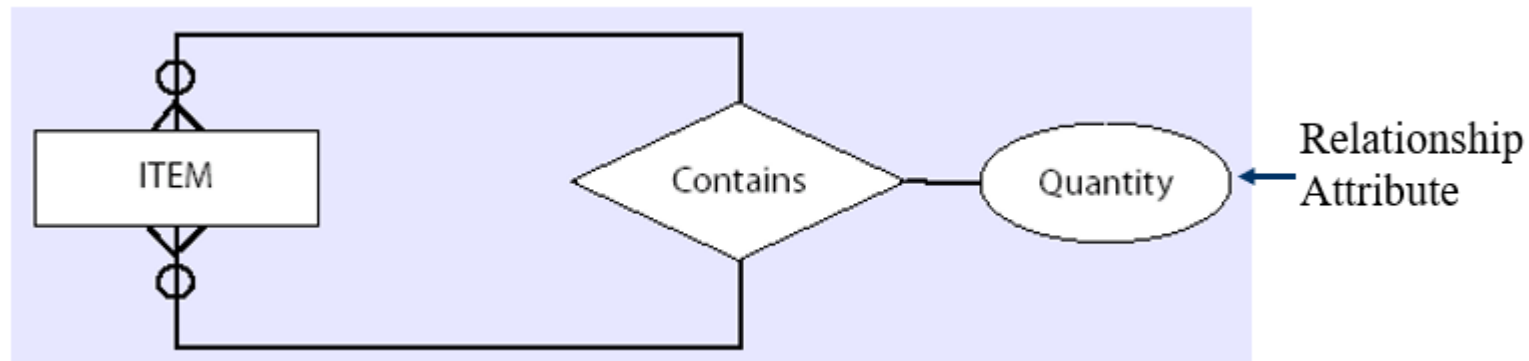
- ❖ Relationship between CUSTOMER and ACCOUNT entities

Cont..

❖ **Attributes of Relationships:** An **attribute on a relationship** stores information related to the relationship.

- Much like attributes on entity types

❖ **Example of an Attribute of a Relationship**



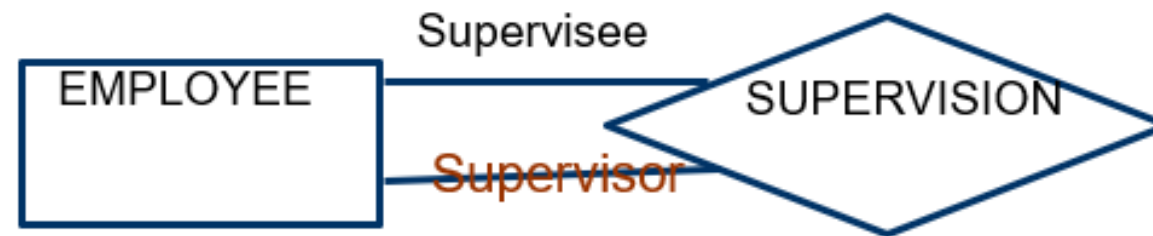
Degree of Relationship

❖ The number of entity sets that participate in a relationship is called the **degree of relationship**.

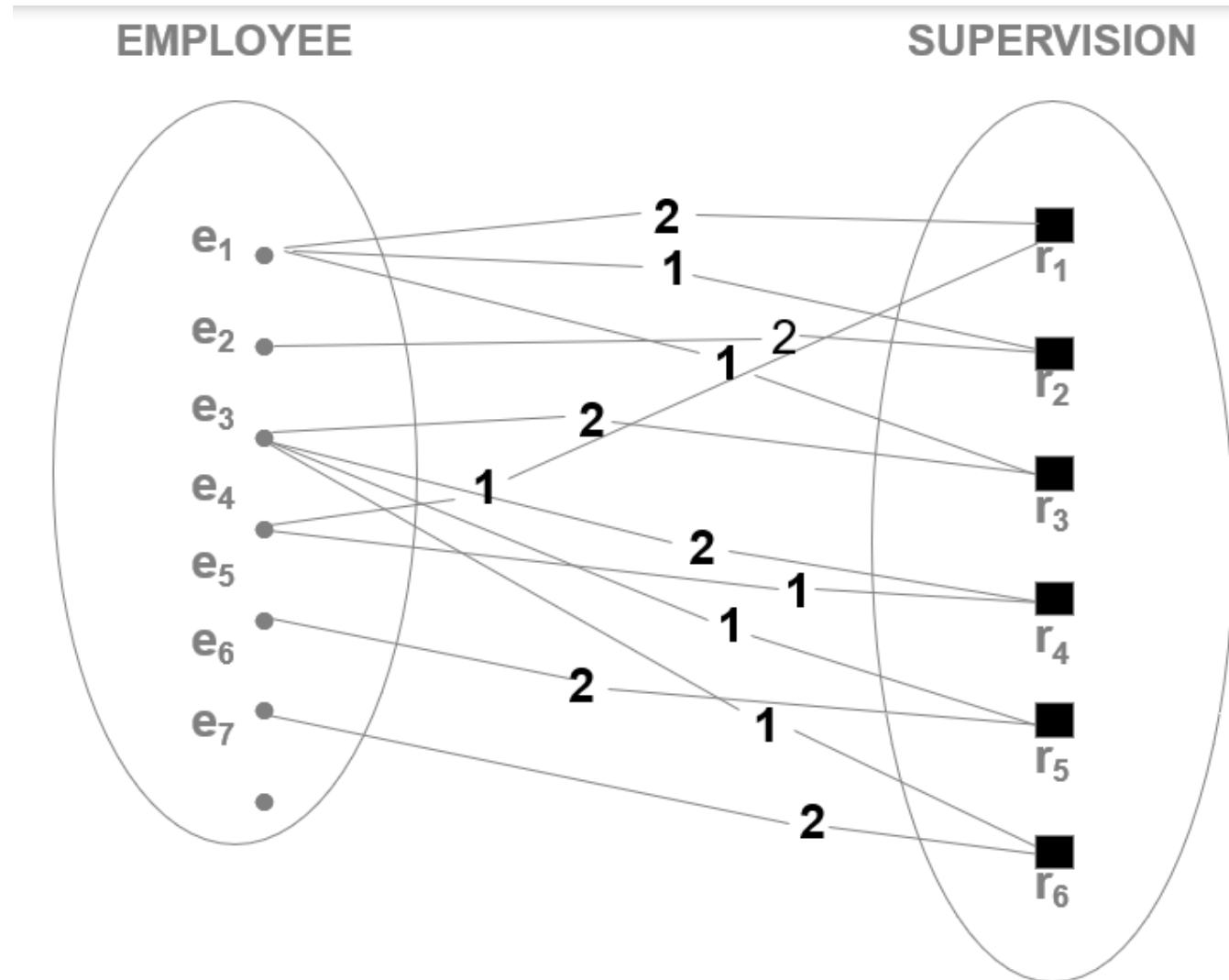
Three common degrees in a database:

❖ Unary (degree 1) – Recursive relationship

- An association between two instances of the same entity type
- $R \in E1 \times E1$
- E.g. An employee supervises another employee



Role names in a recursive relationship



Cont..

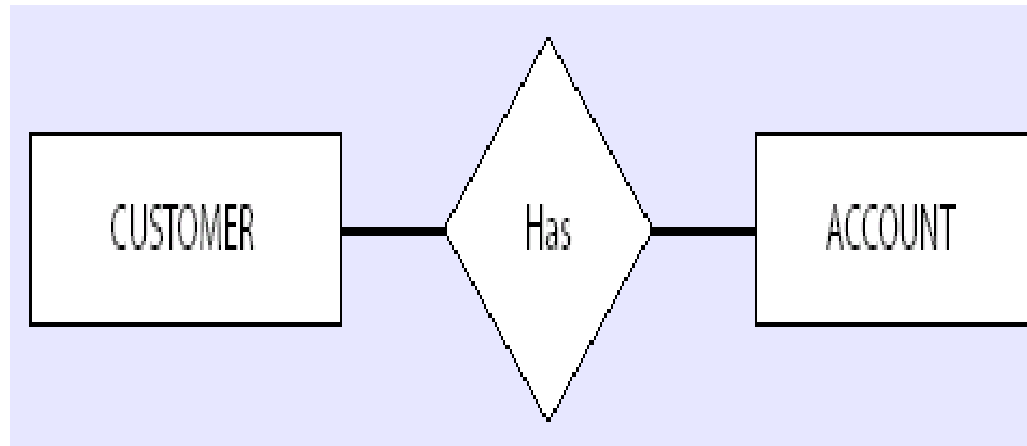
- ❖ **Recursive relationship** is a kind of relationship when an entity type relates to itself.
- ❖ In this case **role names** are essential to distinguish the meaning of each participation in the relationship.
- ❖ In the slide given above the mark “1” represents the Supervisor role , and the mark “2” represents the Supervisee role.

Hence from the diagram in the previous slide we can easily know

- e1 supervises e2 and e3
- e3 supervises e5 and e6

Binary relationship

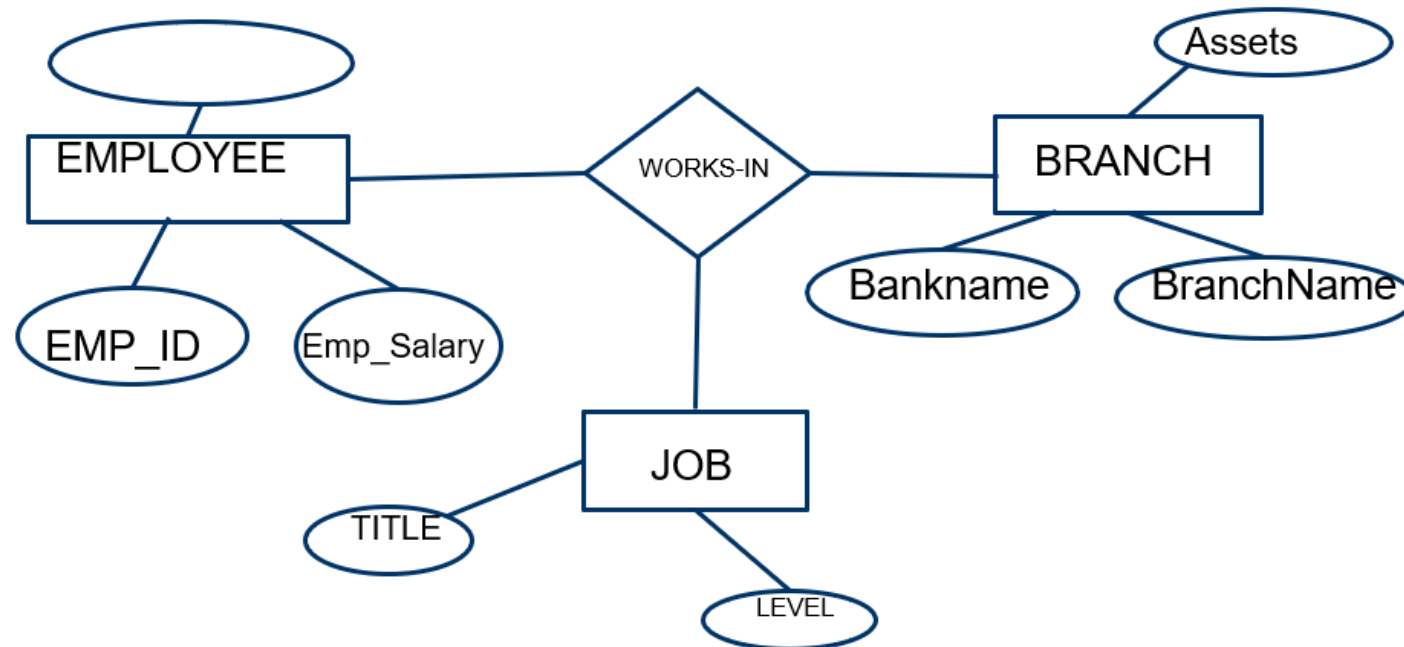
- ❖ **Binary (degree 2)** – Most relationship sets in a DB System are
 - An association between two different entity types
 - $R \in E1 \times E2$
 - E.g. Customer has bank Account



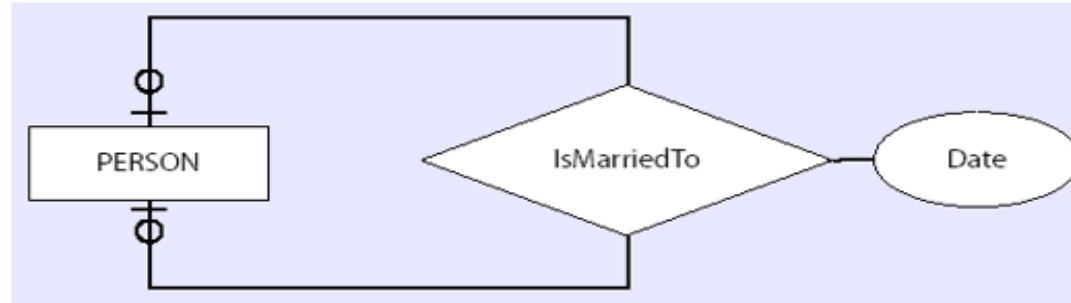
Ternary relationship

❖ Ternary (degree 3)

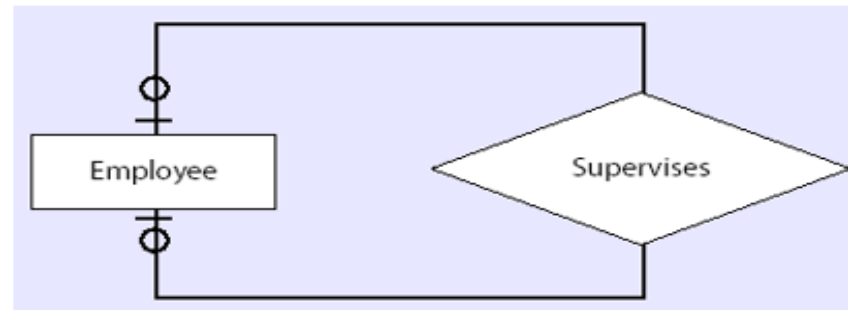
- An association among three different entity types
- $R \in E1 \times E2 \times E3$
- E.g. A bank employee who has a specific job in a bank branch



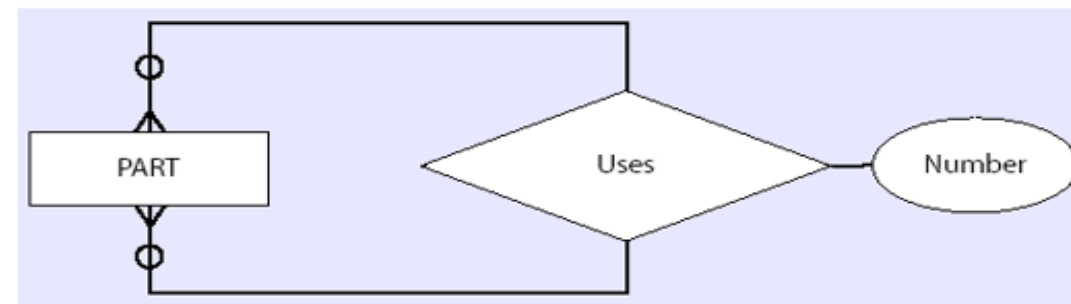
Unary relationship more examples



One-to-One

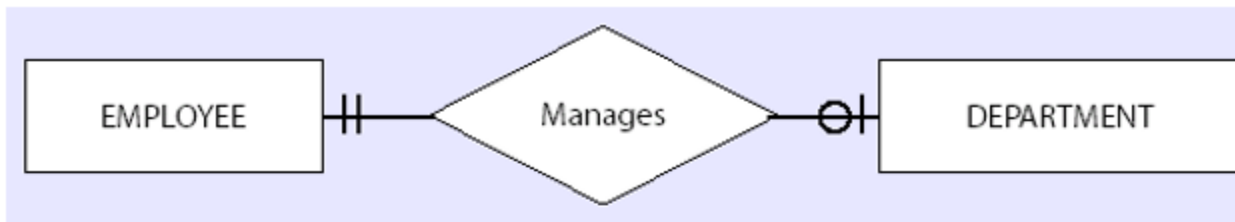


One-to-many

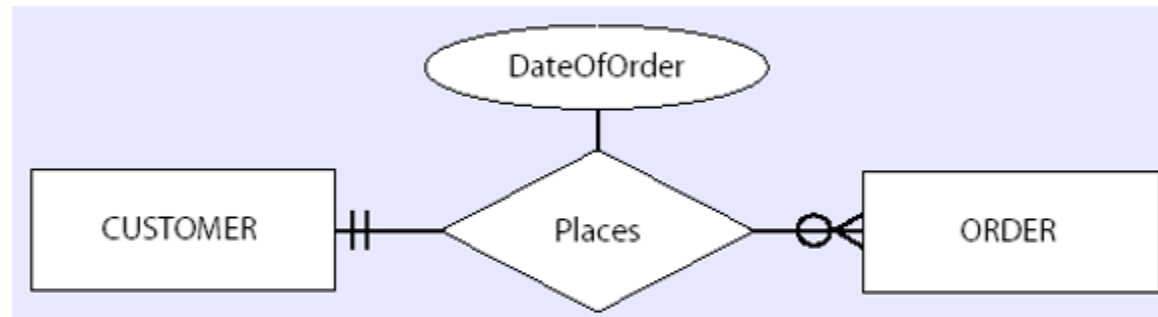


Many-to-Many

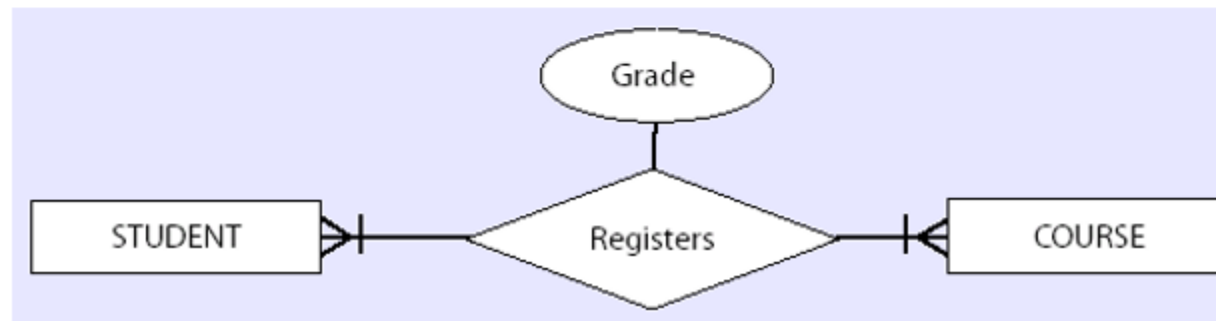
Binary relationship extra examples



One-to-One

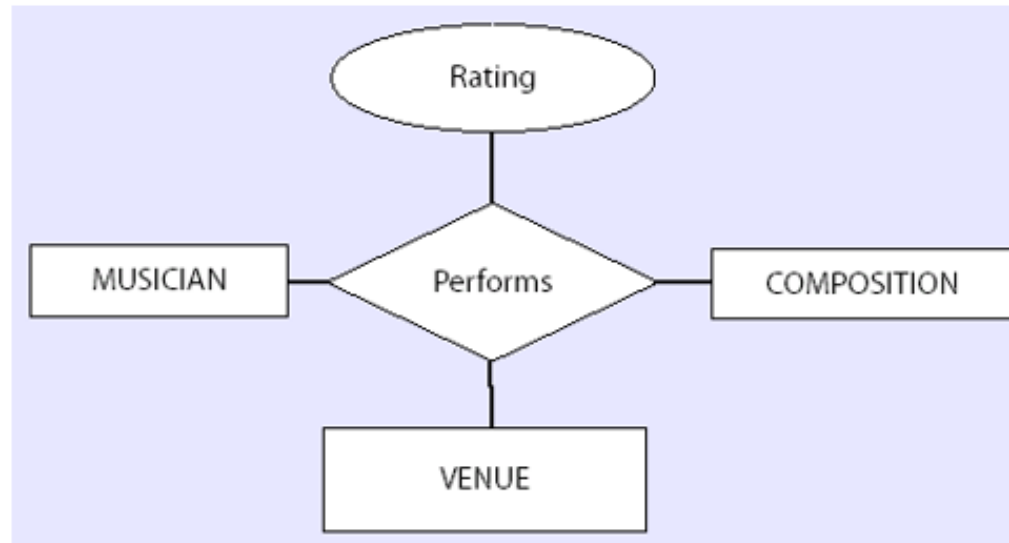


One-to-Many

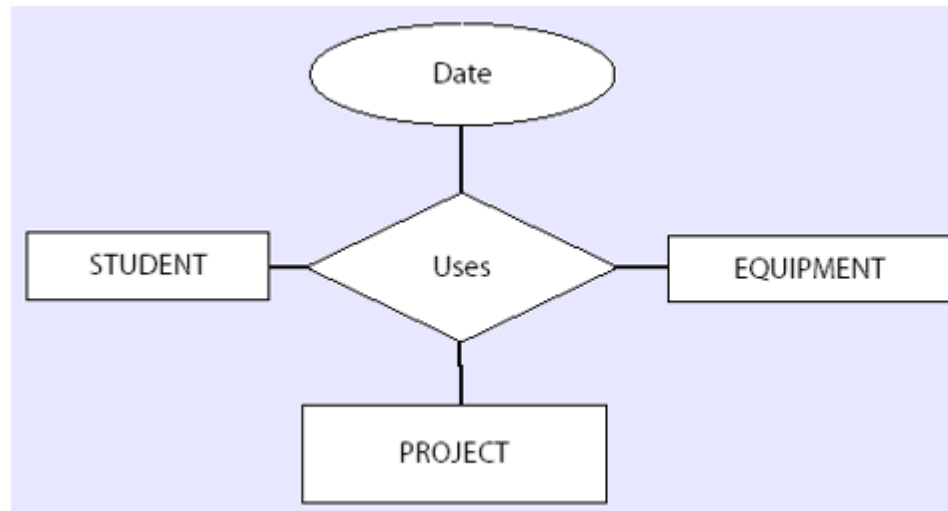


Many-to-Many

Ternary relationship extra examples



Musical Performance Example



Student "Uses" Equipment for a Project Example

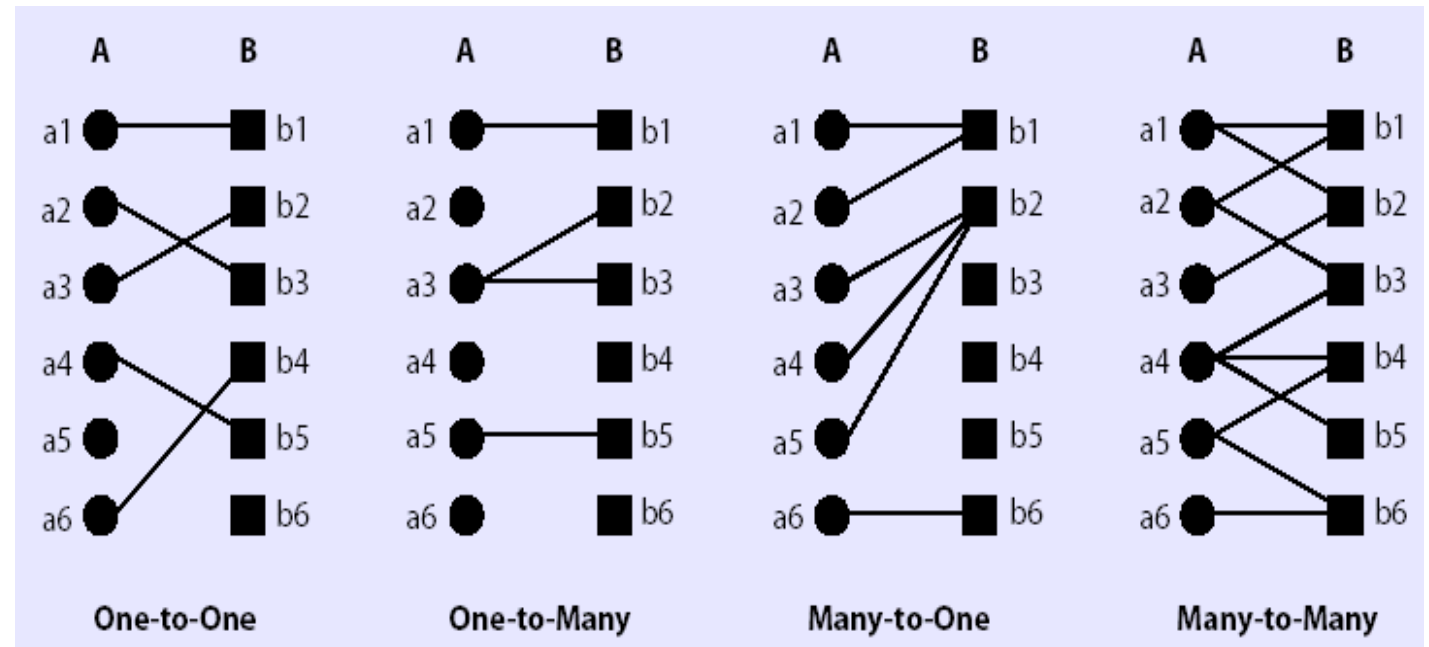
Exercise?: Build an ER Diagram for the following information:

A student record management system will have the following two basic data object categories with their own features or properties: Students will have an Id, Name, Dept, Age, GPA and Course will have an Id, Name, Credit Hours. Whenever a student enroll in a course in a specific Academic Year and Semester, the Student will have a grade for the course.

Constraints on relationship

- ❖ **The cardinality** of a relationship defines the **number** of instances of one entity type that can be associated with instances of another entity type.
- ❖ **The maximum cardinality** of a relationship defines the **maximum number** of instances of one entity type that can be associated with instances of another entity type.

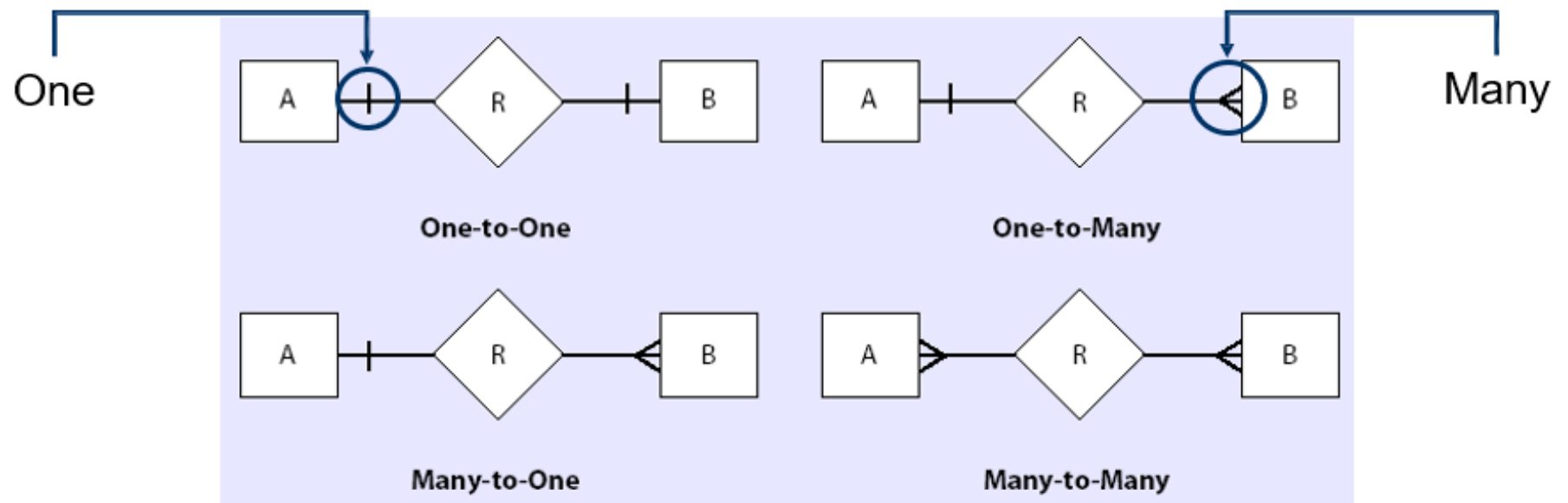
The Four Types of Relationships
between Entity Types



Maximum Cardinality

Relationship types by maximum cardinality:

- ❖ One-to-One
- ❖ One-to-Many
- ❖ Many-to-One
- ❖ Many-to-Many

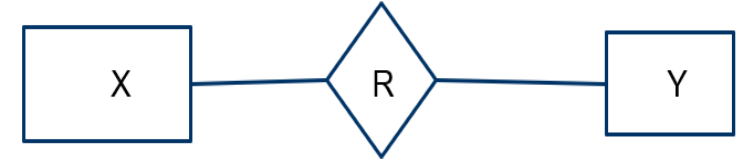


Relationship Types Based on Maximum Cardinality

Cont..

❖ How do you know the maximum cardinality between two entity types?

❖ Let's say we have 2 entity types called X and Y



Two important questions:

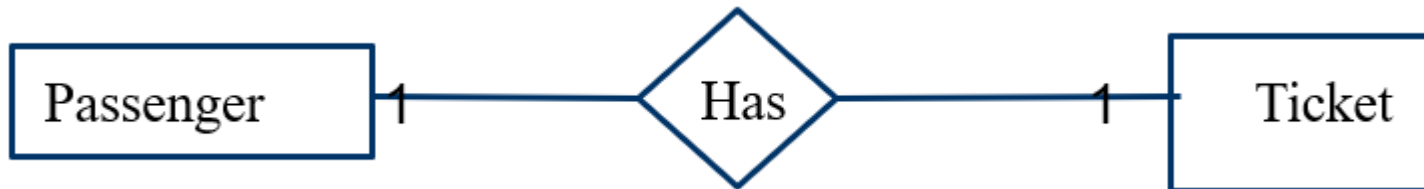
- Q1. Can an instance of X be associated with more than one instance of Y?
- Q2. Can an instance of Y be associated with more than one instance of X?
- The 2 questions above have 4 possible answers. Those answers determine the type of maximum cardinalities

QUESTION 1	QUESTION 2	TYPE OF MAXIMUM CARDINALITY
YES	YES	Many-2-Many
YES	NO	One-2-Many
No	YES	Many-2-One
NO	NO	One-2-One

Maximum Cardinality

❖ One-2-One Relationship:

- An entity in X is associated with **at most one** entity in Y, and vice versa.
- E.g. Each passenger has his own Ticket



❖ One -2-Many Relationship

- An entity in X is associated with any number(zero or more) of entities can associate with at most one entity in X
- E.g. A department contains many employees who work under it



Cont..

❖ Many -2-One Relationship

- An entity in X is associated with at most one entity in Y. But an entity in Y can be associated with any number (zero or more) of entities in X.
- E.g. Many needles can be injected into a single patient

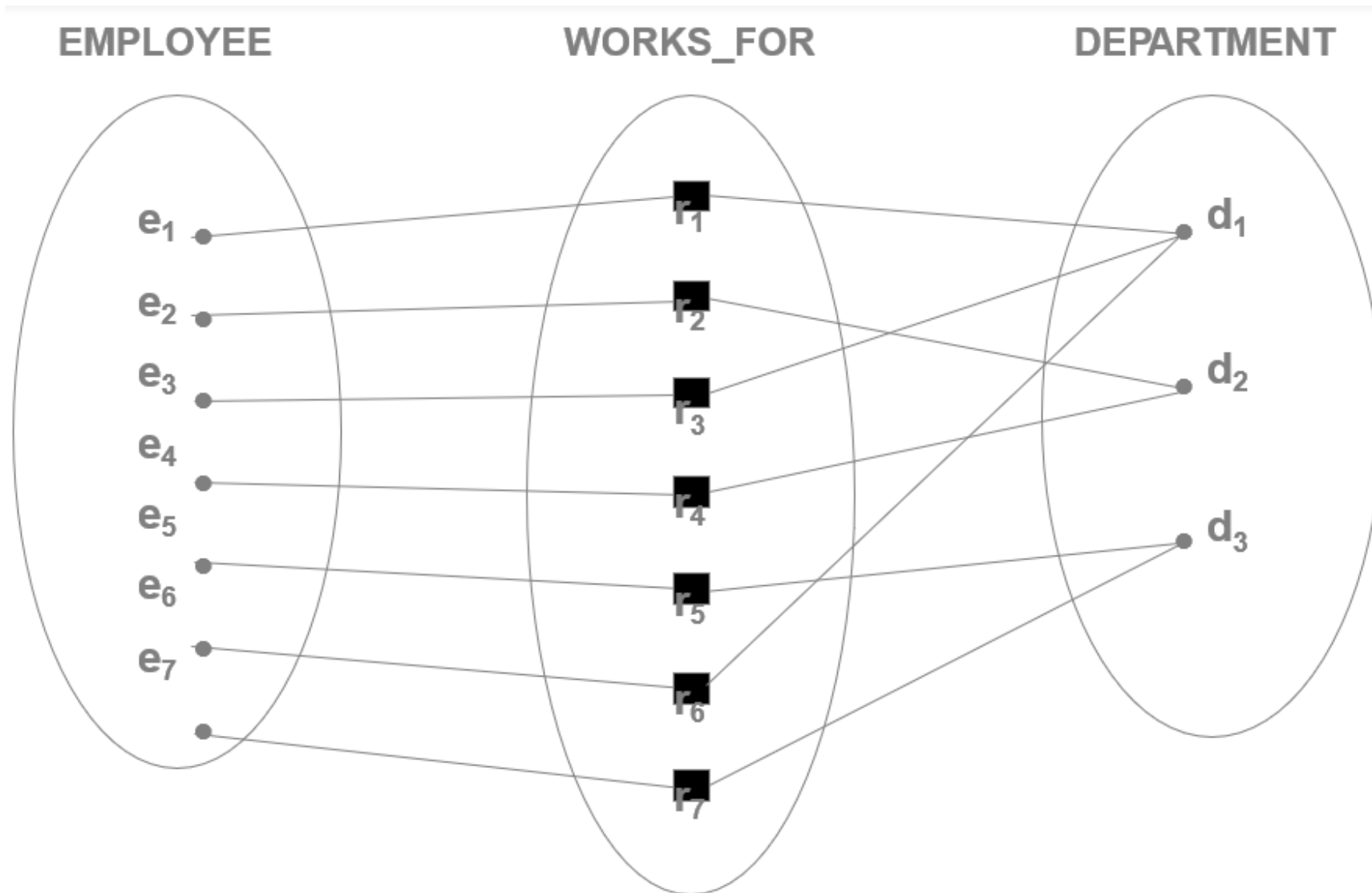


❖ Many-2-Many Relationship

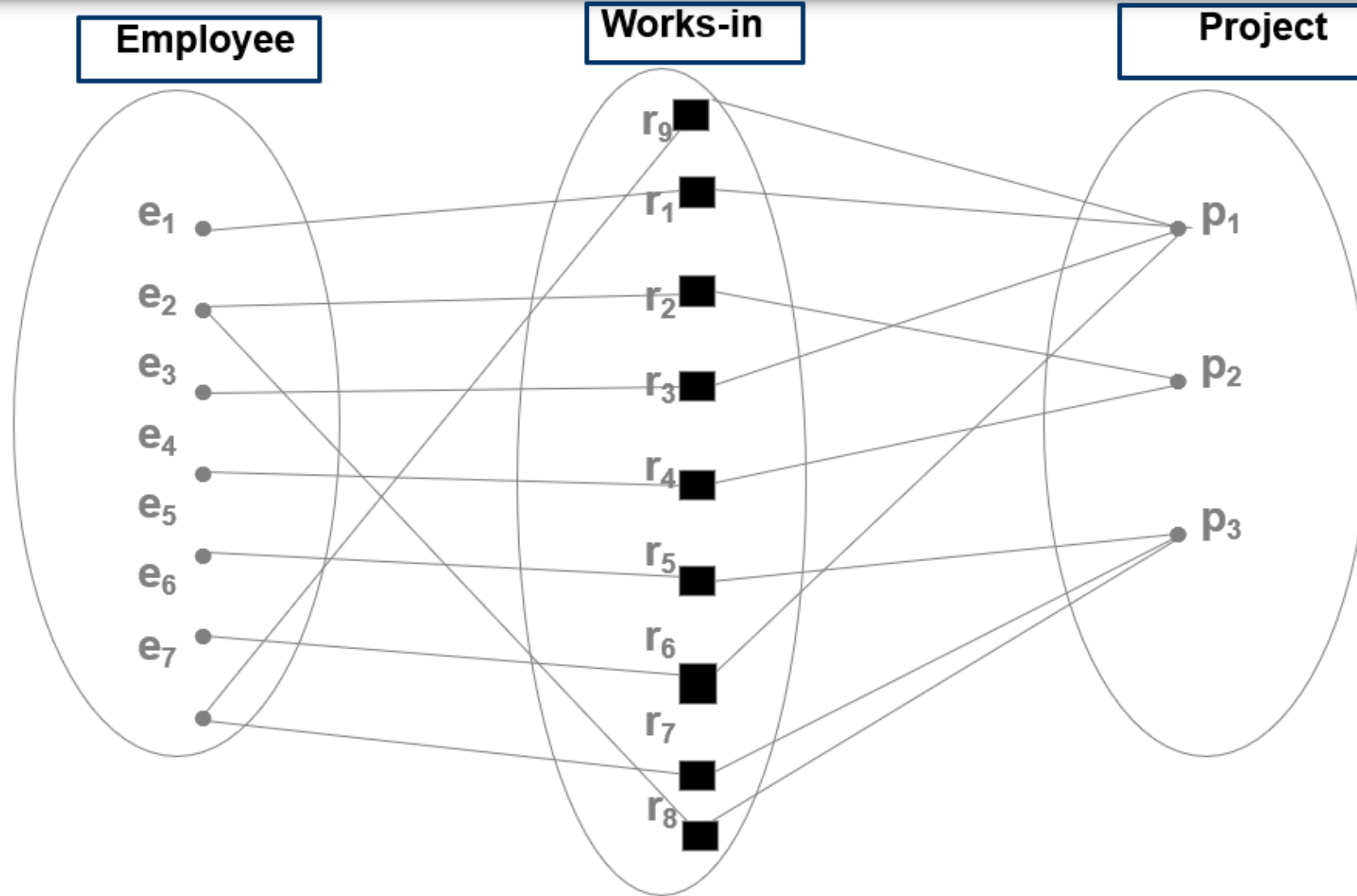
- An entity in X is associated with any number (zero or more) of entities in Y ,and an entity in Y is associated with any number (zero or more) of entities in X



Many to one relationship example



Many to many relationship example



Cardinality Limits of a Relationship: The cardinal limit of a relationship is labeled as:

- ✓ $0..*$ or $0..\infty$ indicating zero or more participation of the entity in the relationship.
- ✓ $1..*$ or $1..\infty$ indicating one or more participation of the entity in the relationship.
- ✓ $0..1$ indicating zero or one participation of the entity in the relationship.
- ✓ $1..1$ indicating exactly one participation of the entity in the relationship.

Cont..

❖ Does the E-R diagram given below provide sufficient information ?

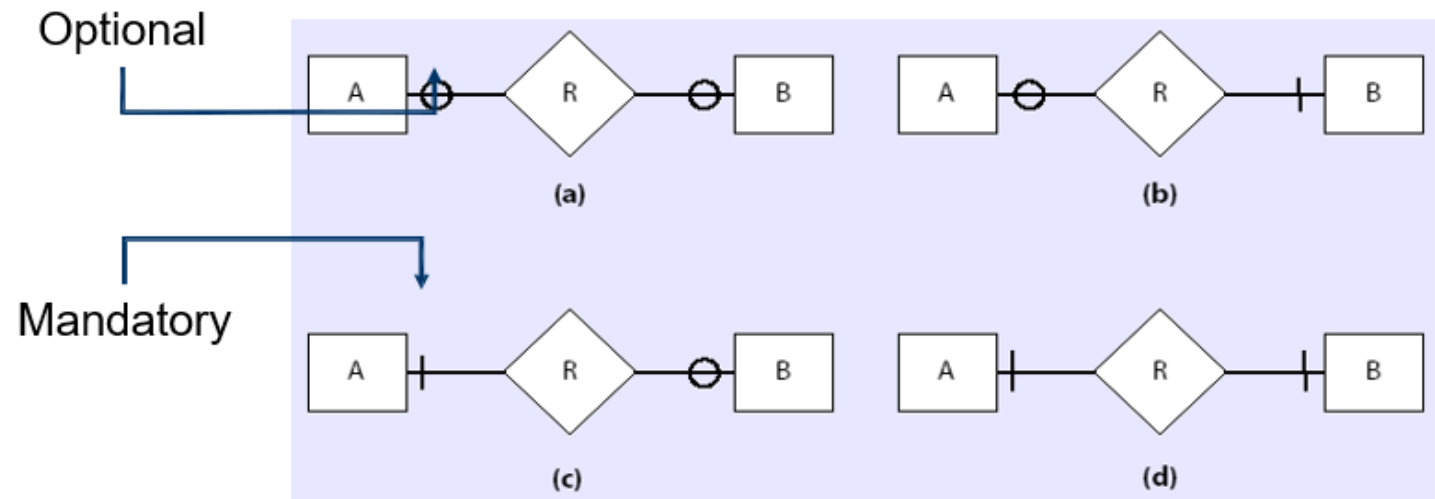
- ✓ What is the minimum number of employees who manage a department?
- ✓ What is the minimum number of departments that are managed by an employee?
- ✓ Is it must for an employee to be a manager of a department?
- ✓ Is it must for a department to have an employee?



Minimum cardinality

- ❖ The **minimum cardinality** of a relationship is defined as the minimum number of instances of one entity type that must be associated with each instance of another entity type.
- ❖ **Minimum cardinality** is also called **participation constraint** or **existence dependency constraints**. There are 2 types of participation constraints:

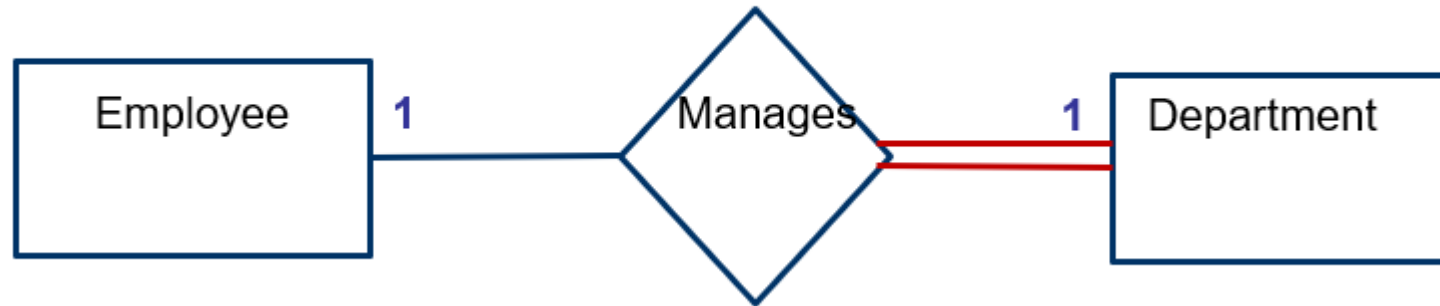
1. Total participation
2. Partial participation



Relationship Types Based on Minimum Cardinality

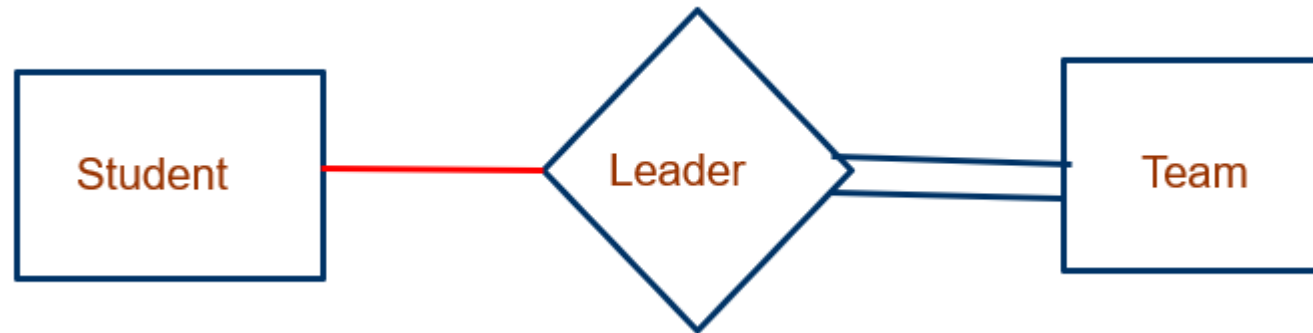
Total Participation

- ❖ In **total participation** , each entity in the entity set occurs in **at least one** relationship in the relationship set.
- ❖ The example below shows that the participation of the entity set Department in the relationship set Manages is total . This is because every department must have a manager .



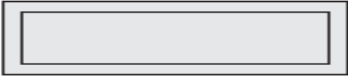
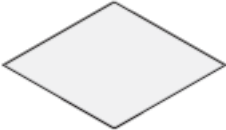
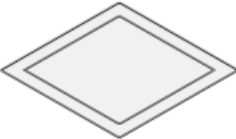
Partial Participation

- ❖ In **partial participation** , some entities (not all entities) in the entity set occurs in the relationship in the relationship set.
- ❖ The example below shows that the participation of the entity set **Student** in the relationship set **Leader** is partial because it is not possible for every student entity to be a leader

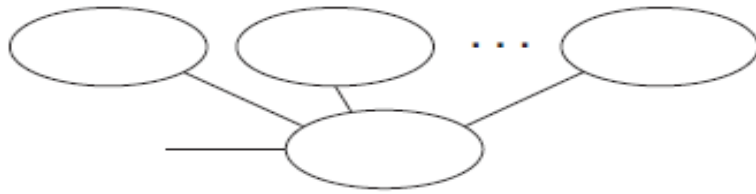


Alternative ER Notations

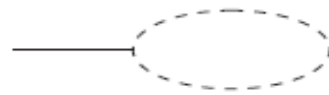
ER Notations:

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute

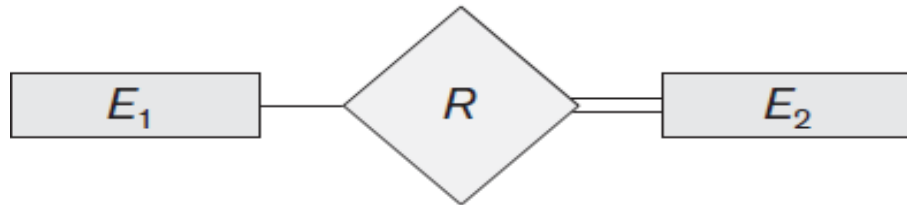
Cont..



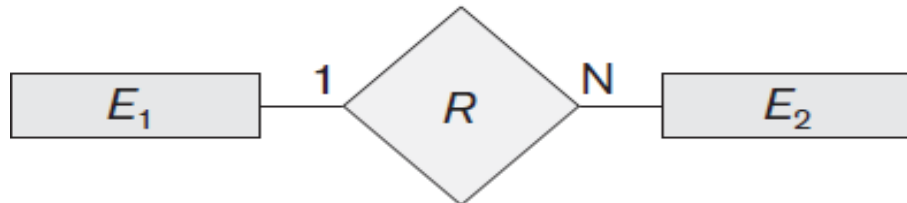
Composite Attribute



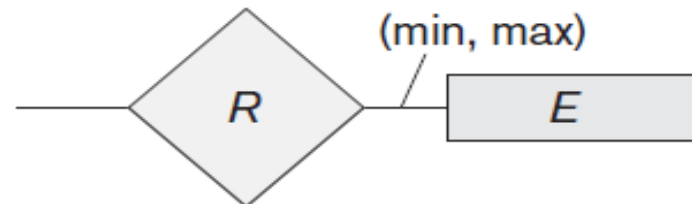
Derived Attribute



Total Participation of E_2 in R

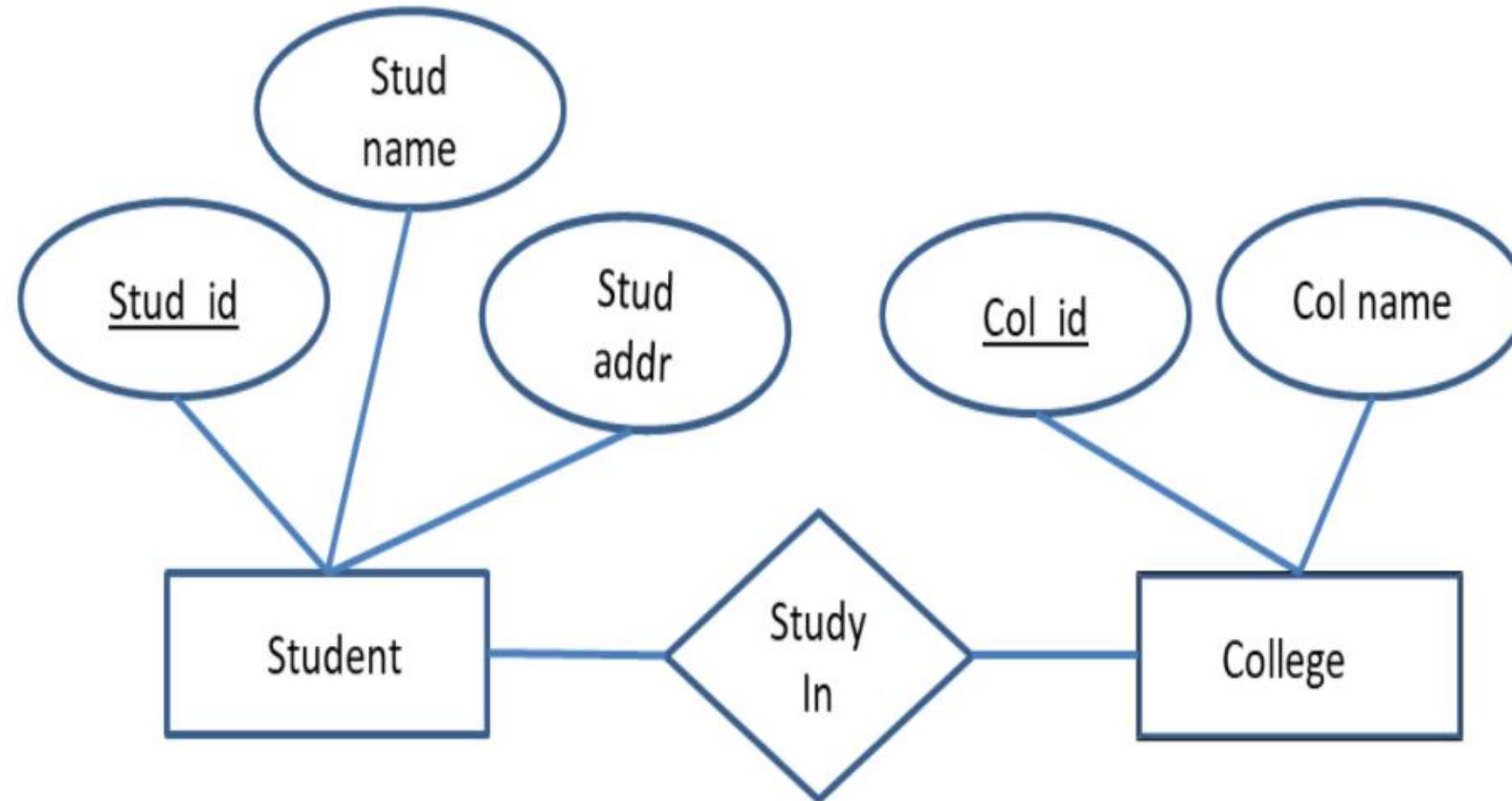


Cardinality Ratio 1: N for $E_1:E_2$ in R



Structural Constraint (min, max)
on Participation of E in R

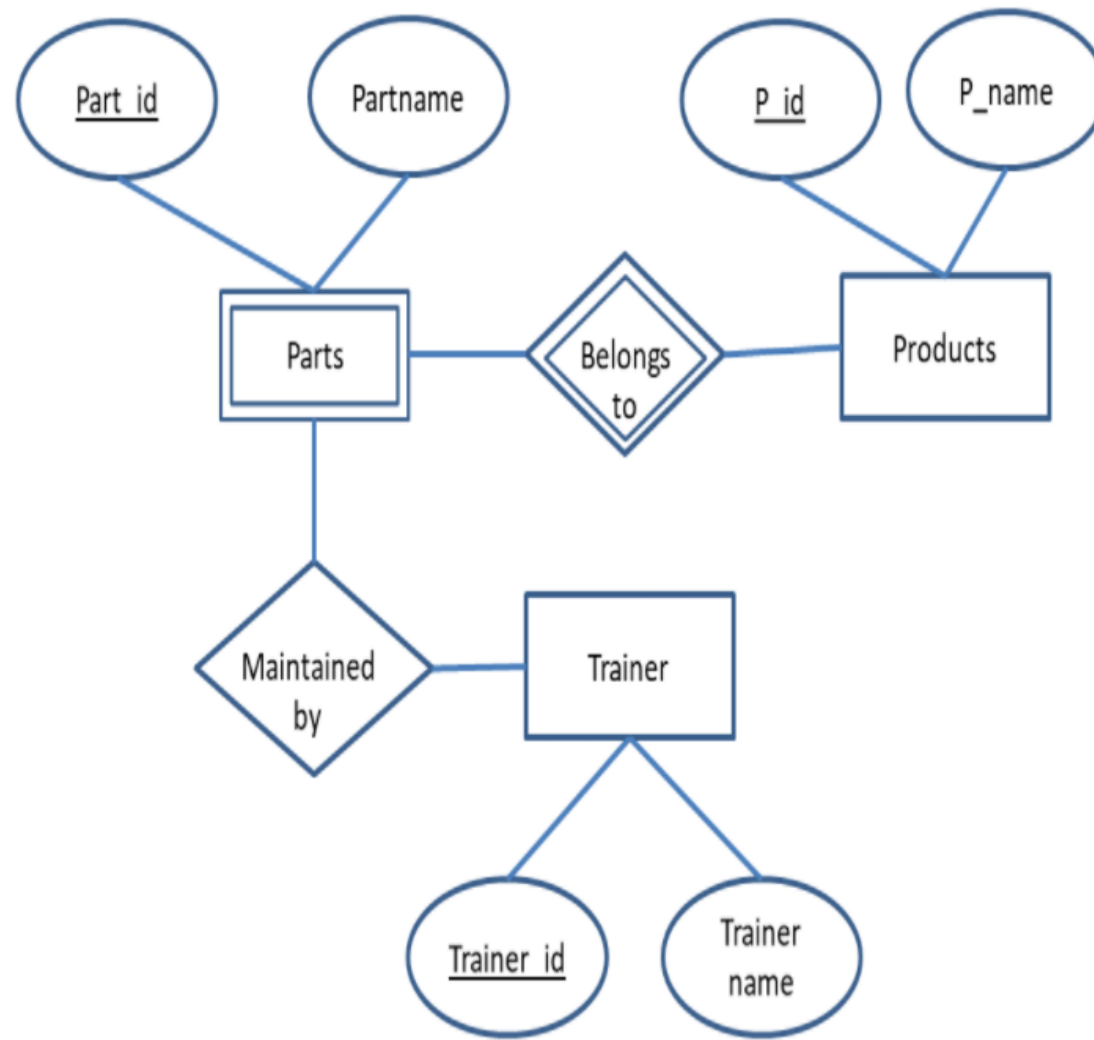
ER Diagram Example (1)



Example 1 (continued)

- ❖ The ER diagram given above (Example 1) shows the relationship named Study In between two entities, Student and College.
- ❖ This is a many to many relationship. Student entity has three attributes, Stud_id, Stud name, Stud addr, which are represents its corresponding properties of Studen entity.
- ❖ Similarly, College entity has Col_id for college id and Col name for the name of the college.
- ❖ Stud_id is the key attribute of Student entity and Col_id is the key attribute of College entity.

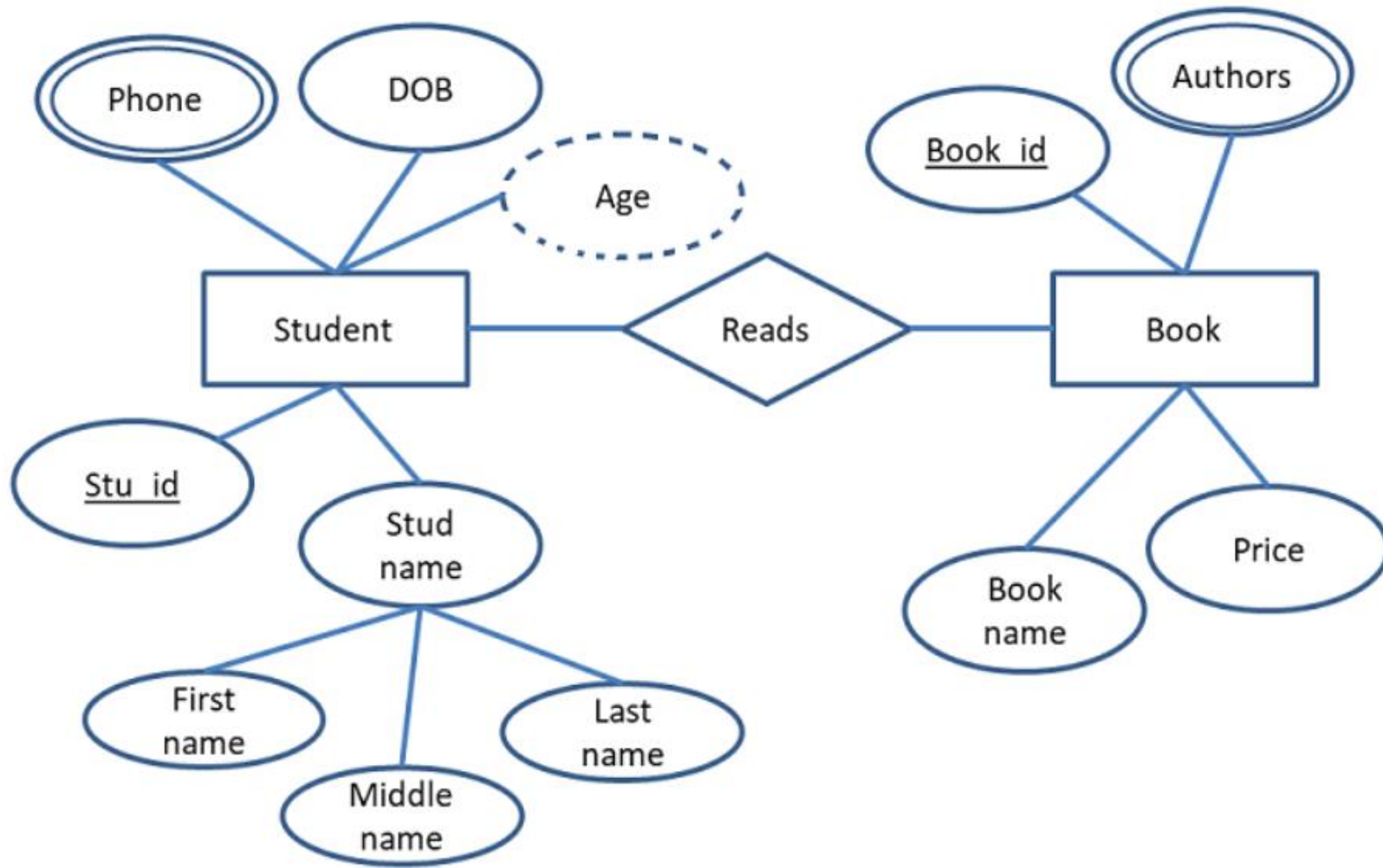
Example (2)



Example 2 (continued..)

- ❖ In the ER diagram (Example 2), Products, trainers are the strong entities and the Parts entity is a weak entity.
- ❖ Because Parts entity depends on Products entity on a relationship of Belongs to for its existence.
- ❖ And also parts entity related with trainer entity through a relationship named maintained by part_id and part name are two attributes of parts entity and its key attributes are part_id.
- ❖ Part_id and part name are the attributes of the products entity and its key attribute is part-id.
- ❖ Similarly, trainer_id and trainer name are the attributes of a trainer and its key attribute is trainer_id.

Example 3



Example 3 (continued..)

- ❖ In the ER diagram (example 3), some important types of attributes are specified.
- ❖ Reads are the relationship between two entities namely student and book.
- ❖ The student entity has a key attribute of stud_id, a composite attribute of stud_name with the components such as first name, middle name, and last name., multivalued attribute f phone and derived attribute of age with its base attribute of DOB.
- ❖ Similarly, the book entity has a key attribute of book_id, a multivalued attribute of authors, and other two attributes such as book_name and price.

Database Application example (Company database)

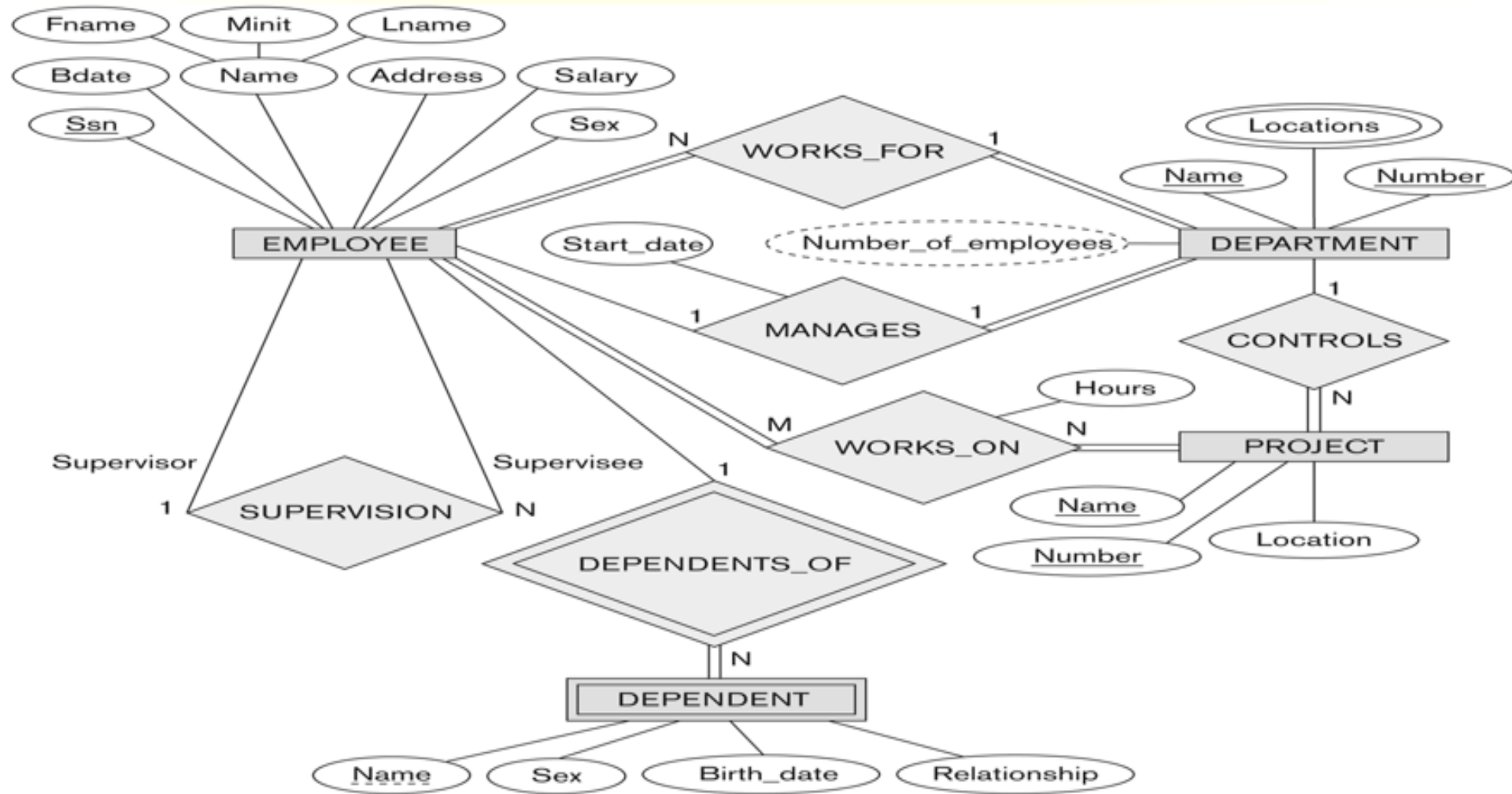
Company Database

- ❖ We need to create a database schema design based on the following (simplified) **requirements** of the **COMPANY Database**:
 - ✓ The company is organized into **DEPARTMENTS**. Each department has a *name*, *number* and *an employee* who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
 - ✓ Each department *controls* a number of **PROJECTS**. Each project has a *unique name*, *unique number* and is located at a *single location*.

Company database example (continued)

- ❖ We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
 - ✓ Each employee *works for* one department but may *work on* several projects.
 - ✓ We keep track of the number of hours per week that an employee currently works on each project.
 - ✓ We also keep track of the *direct supervisor* of each employee.
- ❖ Each employee may *have* a number of DEPENDENTS.
 - ✓ For each dependent, we keep track of their name, sex, birthdate, and relationship to the employee.

ER Diagram (For the company database schema)



Keys and Constraints

Some of the commonly used classifications of database constraints are:

- 1. Keys** – are attributes or set of attributes that can be used to uniquely identify an entity within the entity set. A key in E/R model is represented by *underlining* the attributes belonging to the key
- 2. Single-value constraints** – are requirements that specify a given attribute or set of attributes are unique in certain context.
- 3. Referential integrity constraints** – are requirements for an existence of an entity in the database so as it can be referenced by a certain relationship. Referential integrity in E/R model is represented by a *rounded arrow head* pointing to the entity set required for existence.

4. Domain constraints – are requirements on an attribute value to be in a specified range of values. No specific notation, but side marks may be used to represent domain constraints.

5. Participation constraints – are assertions whether the participation of an entity in a relationship is total or partial.

- ❖ **Keys** as described above **keys** are attributes or set of attributes that suffice to distinguish entities from each other.
- ❖ A **super key** also known as **super set** is then a set of one or more attributes that in group (collectively) can identify an entity uniquely from the entity set.
- ❖ Example: Consider the “EMPLOYEES” entity set, then
- ❖ - “EmpId”, “EmpId, Name”, “NationalId”, “NationalId, BDate”, ... are super keys
- ❖ “Name”, “BDate” are NOT super keys

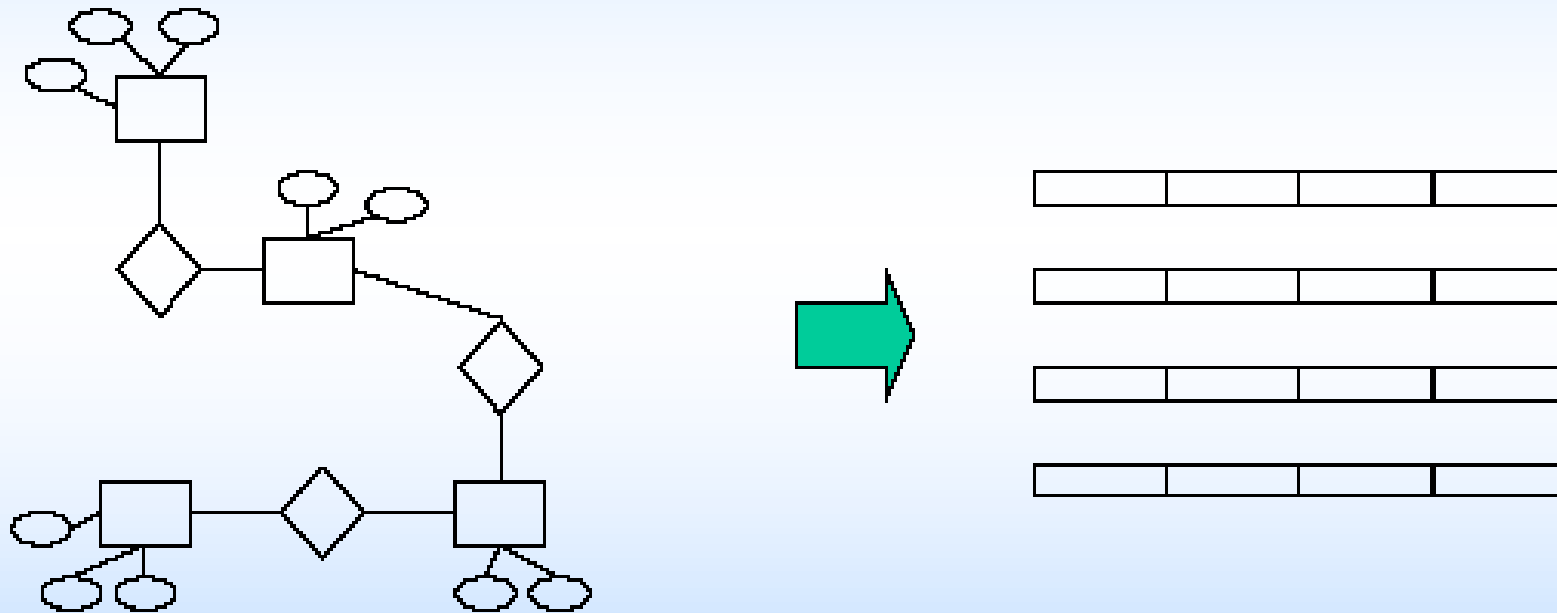
❖ A **candidate key** is an attribute (or set of attributes) that uniquely identifies a row. A candidate key must possess the following properties:

- Unique identification - For every row the value of the key must uniquely identify that row.
- Non redundancy - No attribute in the key can be discarded without destroying the property of unique identification.

- ❖ A **primary key** is the candidate key which is selected as the principal unique identifier. Every relation must contain a primary key. The primary key is usually the key selected to identify a row when the database is physically implemented
- ❖ A **foreign key** is an attribute (or set of attributes) that appears (usually) as a non key attribute in one relation and as a primary key attribute in another relation

Mapping ER Diagram to Relations

ER/EER to database schema



What is well structured relation?

- ❖ A relation that contains a minimum amount of redundancy and
- ❖ Allows users to insert, modify and delete the rows without errors or inconsistencies.

EMPLOYEE1

<u>Emp_ID</u>	Name	Dept	Salary
100	Margaret Simpson	Marketing	42,000
140	Allen Beeton	Accounting	39,000
110	Chris Lucero	Info Systems	41,500
190	Lorenzo Davis	Finance	38,000
150	Susan Martin	Marketing	38,500

Steps to transform ER model into Relations

❖ It is useful to transform the conceptual data model into a set of normalized relations.

❖ Steps:

- i. Represent entity types as relations and their attributes as the column.
- ii. Represent relationships in an appropriate way
- iii. Normalize the relations if required
- iv. Merge the relations if required

Example

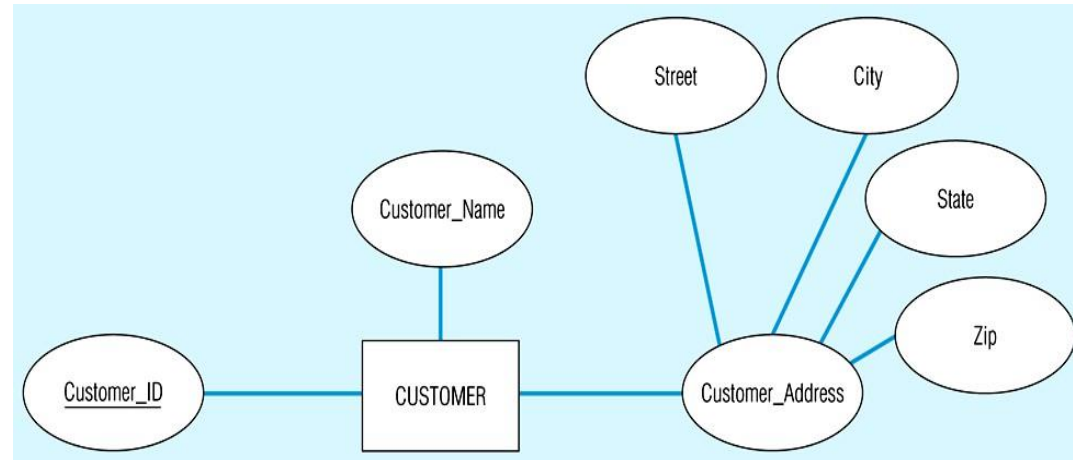


CUSTOMER

<u>Customer_ID</u>	Name	Address	City_State_ZIP	Discount
1273	Contemporary Designs	123 Oak St.	Austin, TX 28384	5%
6390	Casual Corner	18 Hoosier Dr.	Bloomington, IN 45821	3%

Converting regular entities into relations

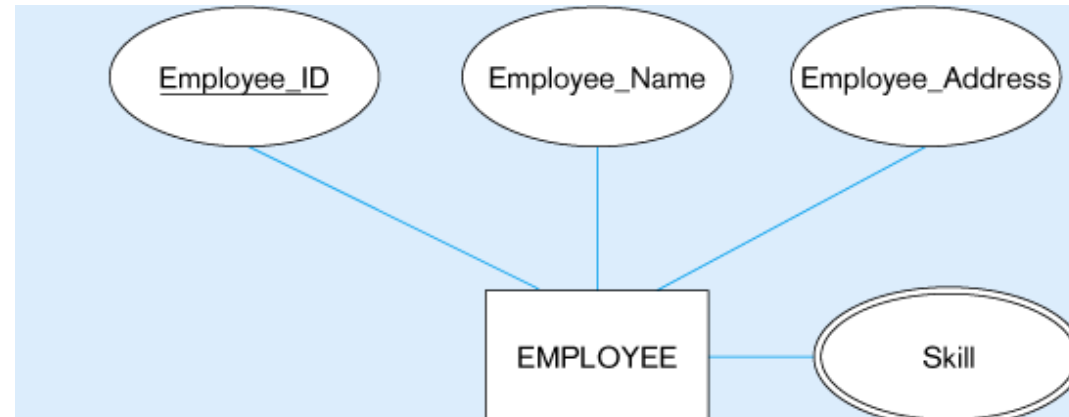
- ❖ Composite attributes: Use only their simple, component attributes
- ❖ Multi-valued Attribute - Becomes a separate relation with a foreign key taken from the superior entity.



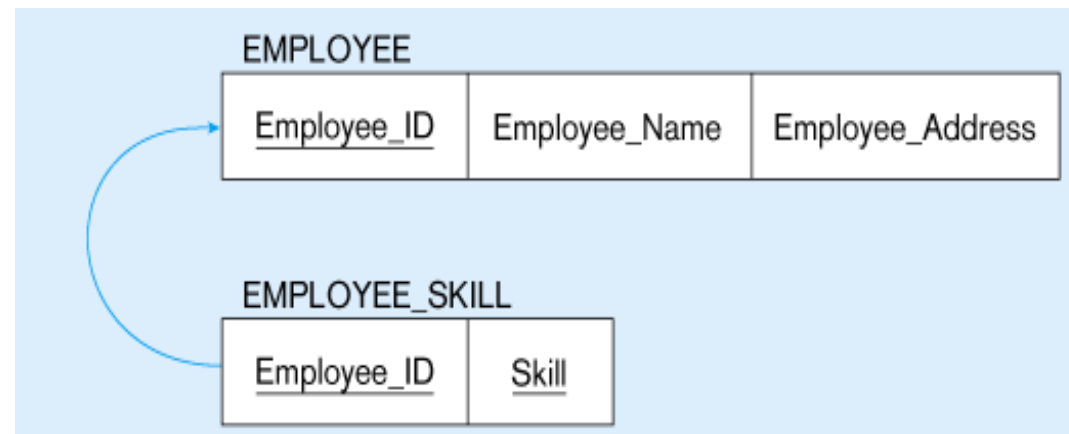
CUSTOMER					
<u>Customer_ID</u>	Customer_Name	Street	City	State	Zip

Cont..

a. Multivalued attribute becomes a separate relation with foreign key.



b. 1:M relationship between original entity and new relation

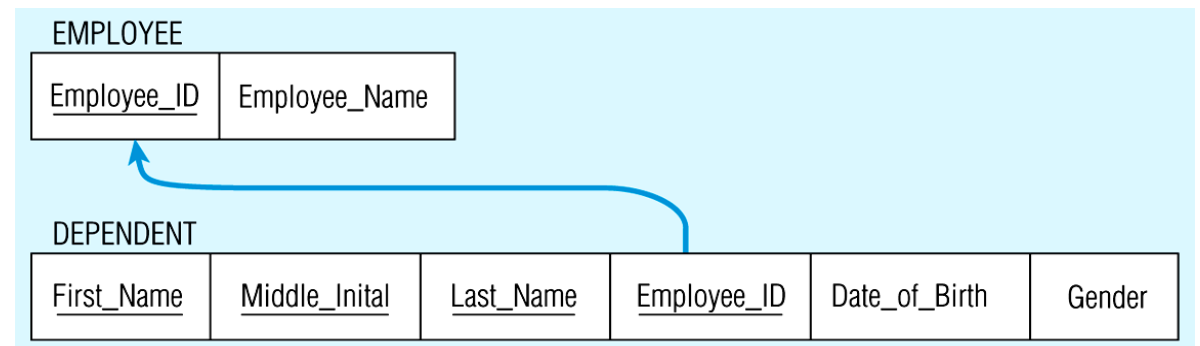
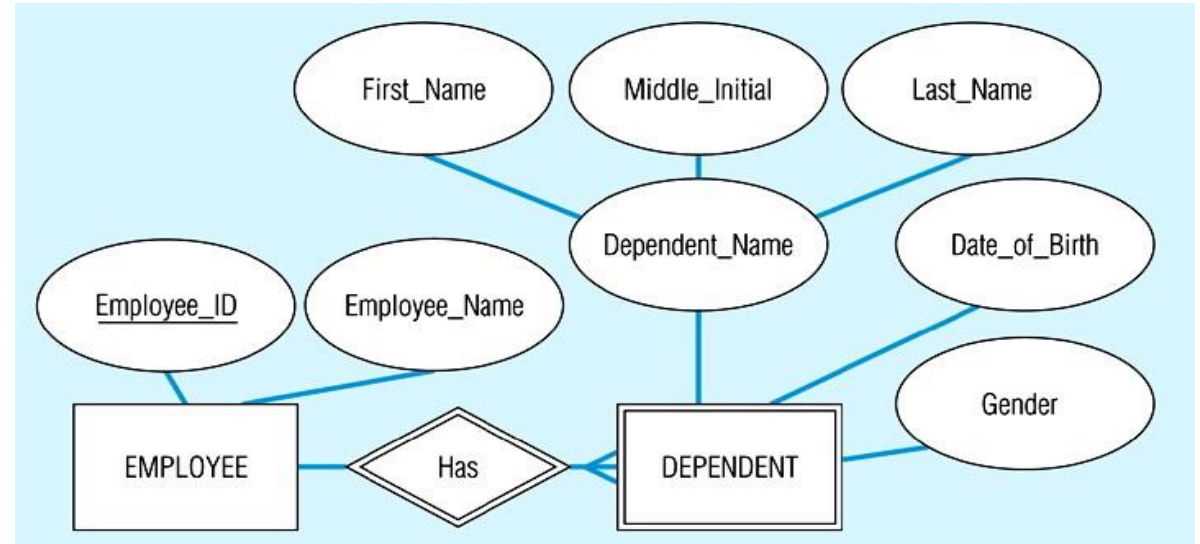


Converting weak entity types into relations

Becomes a separate relation with a foreign key taken from the owner entity type.

❖ Primary key composed of:

- Partial identifier of weak entity
- Primary key of strong entity



Converting Unary Relationships into Relations

❖Unary 1:N Relationships

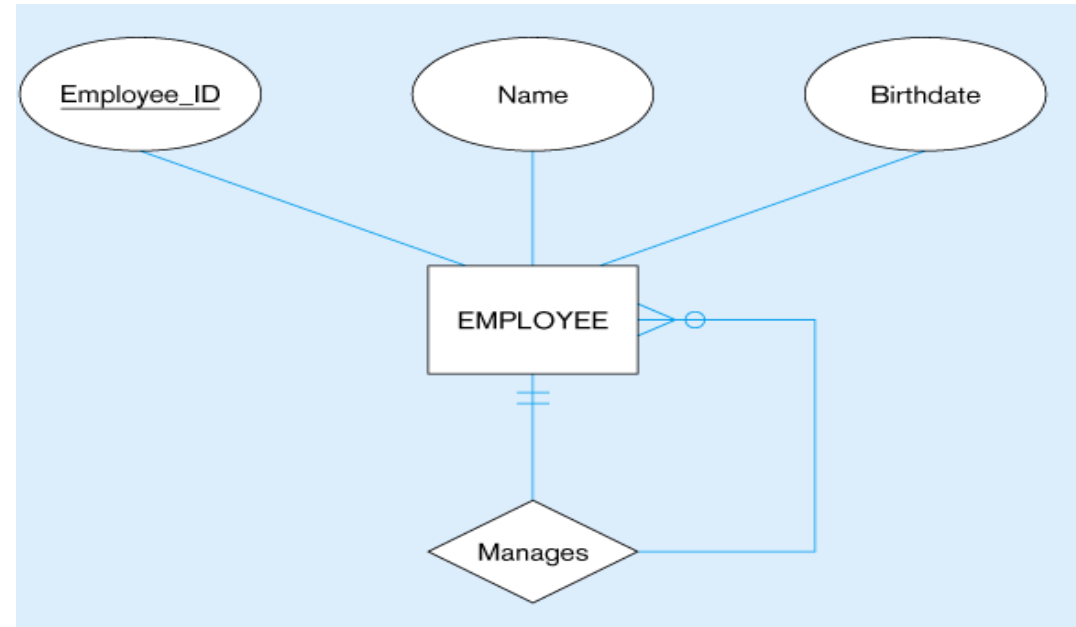
- Relationship between instances of a single entity type
- Utilize a recursive foreign key
- A foreign key in a relation that references the primary key values of that same relation

❖Unary M:N Relationships

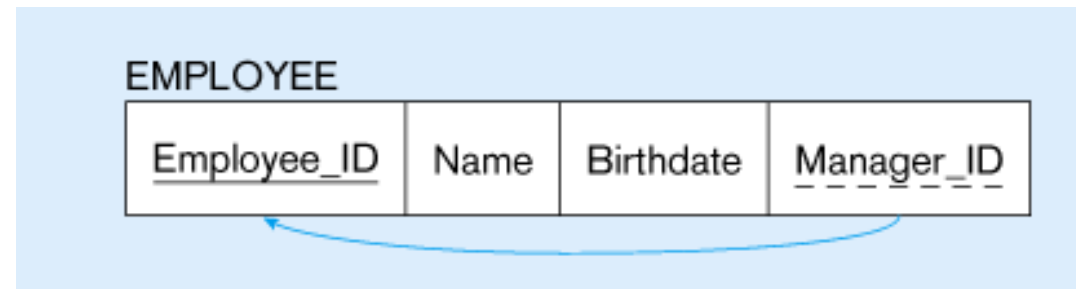
- Create a separate relation
- Primary key of new relation is a composite of two attributes that both take their values from the same primary key

Cont..

Mapping a Unary 1:N
Relationship



EMPLOYEE relation with
recursive foreign key

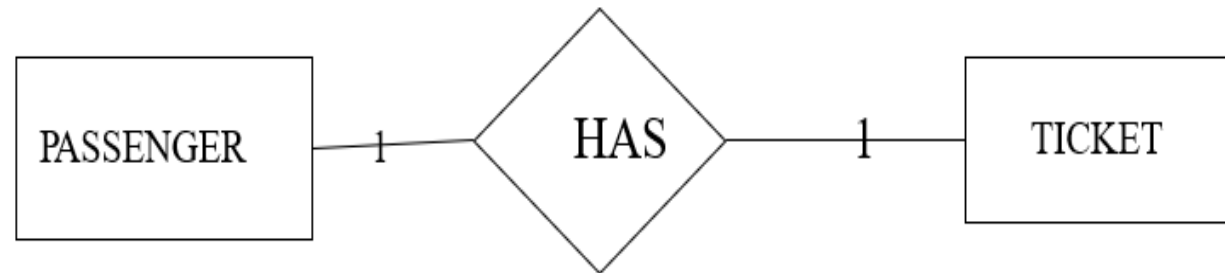


Converting Binary Relationships into Relations

❖ Binary 1:1

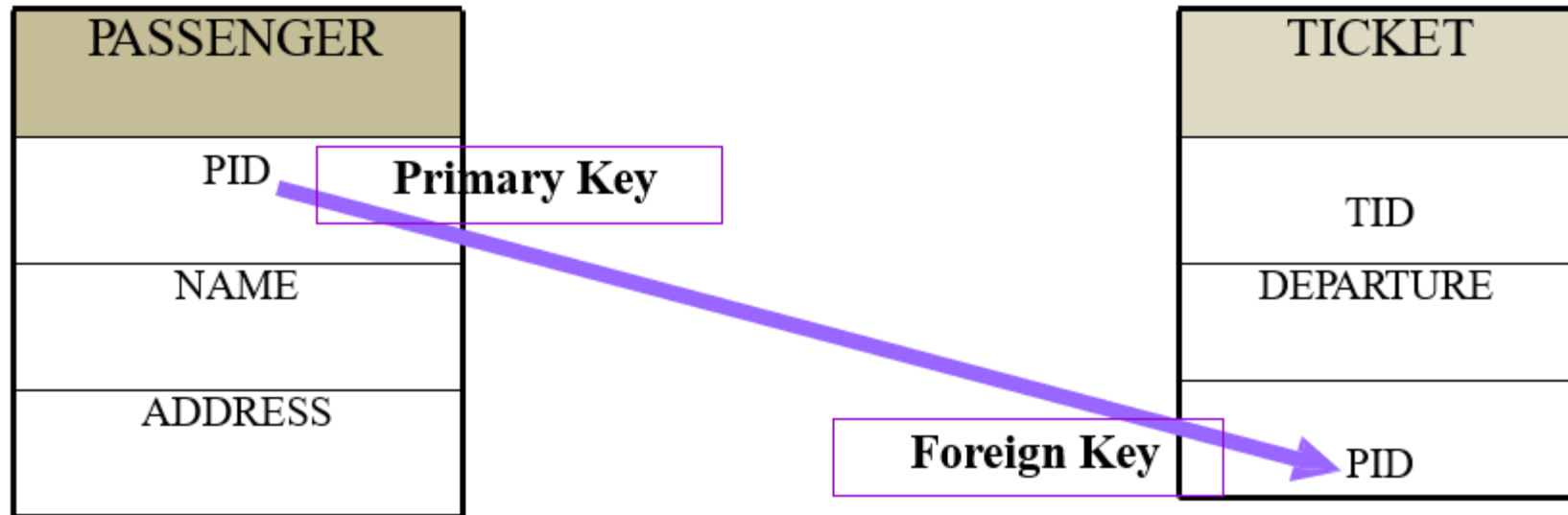
Three possible options:

- a. Add the primary key of A as a foreign key of B
- b. Add the primary key of B as a foreign key of A
- c. Both



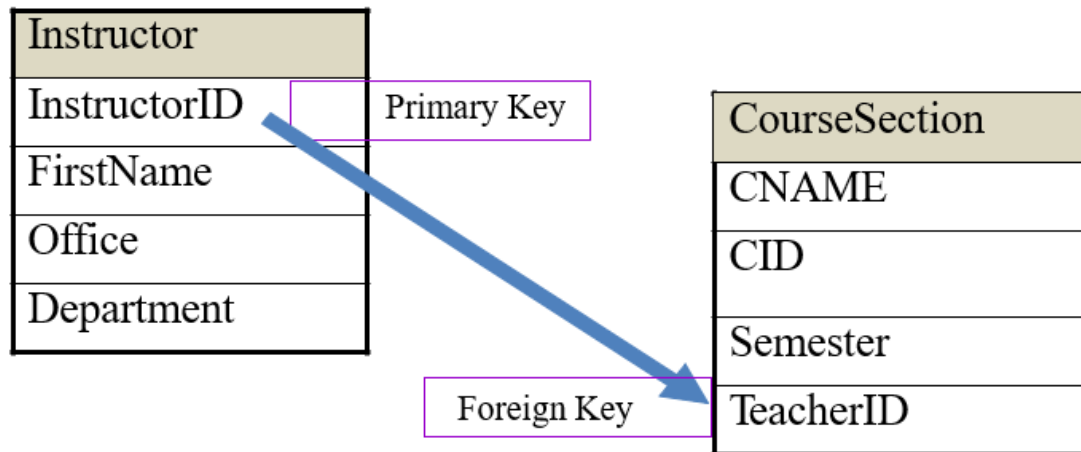
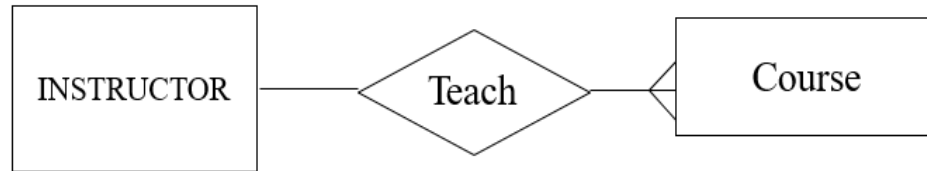
Cont..

❖ In an One-to-One relationship, two table to avoid redundancy issues



❖ Binary 1:N Relationships

- Add the primary key attribute (or attributes) of the entity type on the “ONE” side of the relationship as a foreign key into the relation on the “MANY” side.
- The primary key on the “one” side migrates to the “many” side . The foreign key is always added into the “Many” side



❖ **Binary M:N relationships:**

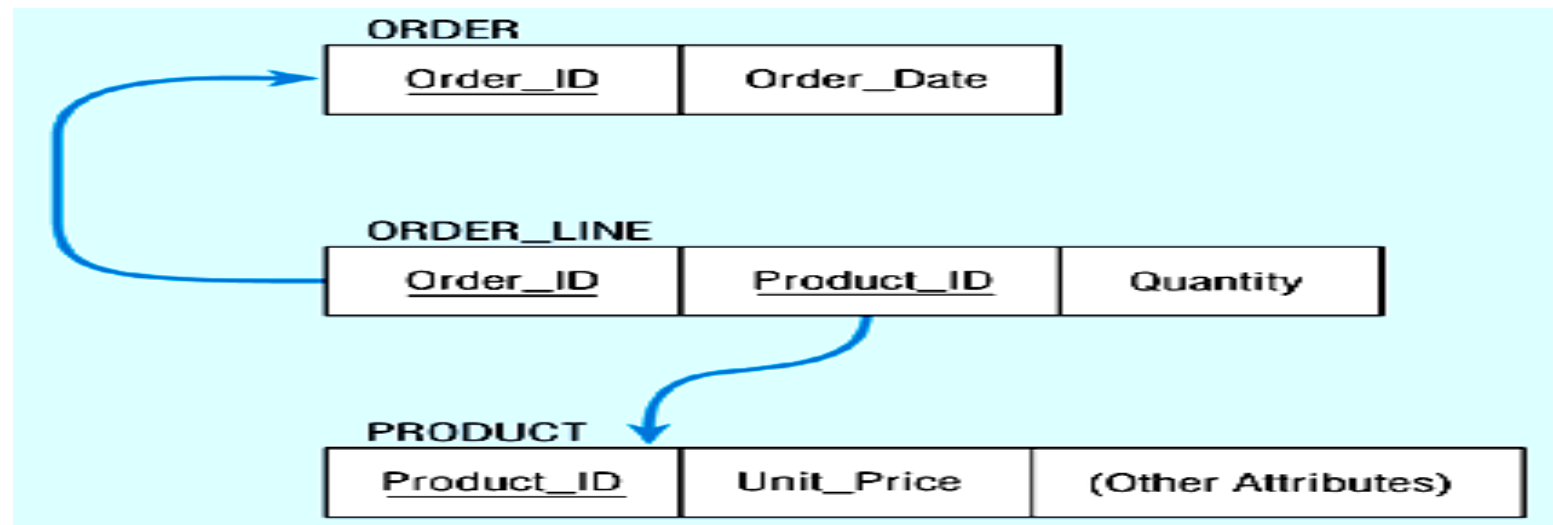
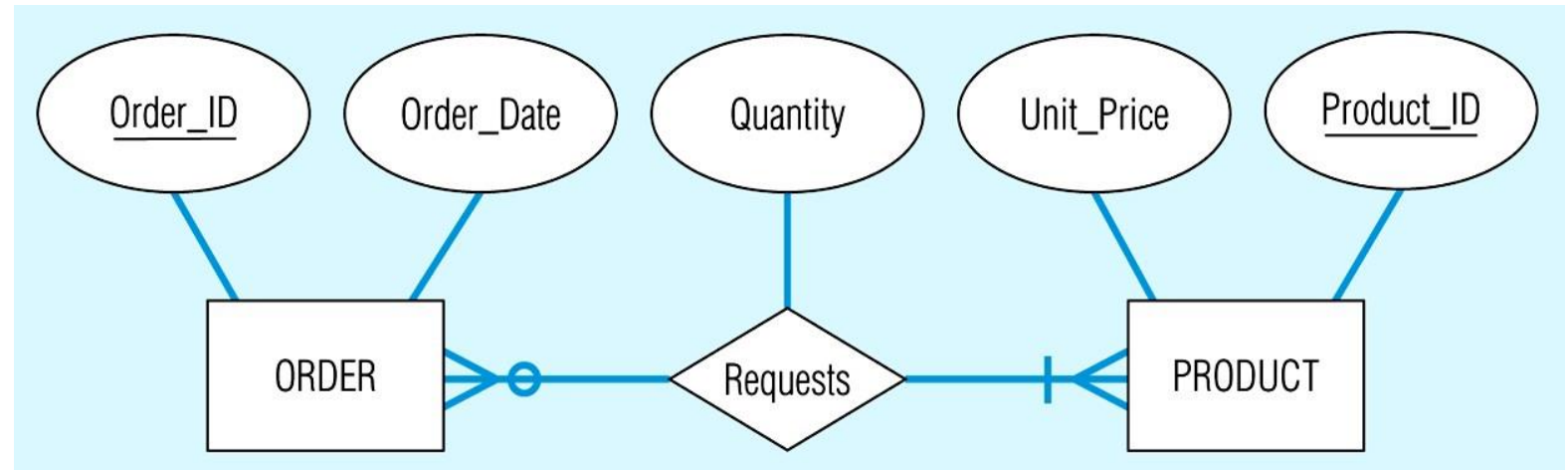
- ✓ Create a new table that has a primary key which is the combination of the primary keys of all the participating entity types. Those added primary keys also act as foreign keys in the new table.
- ✓ All descriptive attributes of the relationship type among the participating entity types must also be included in the new table.

The primary key must satisfy the following two conditions

- ✓ The value of the key must uniquely identify every row relation
- ✓ The key should be non redundant

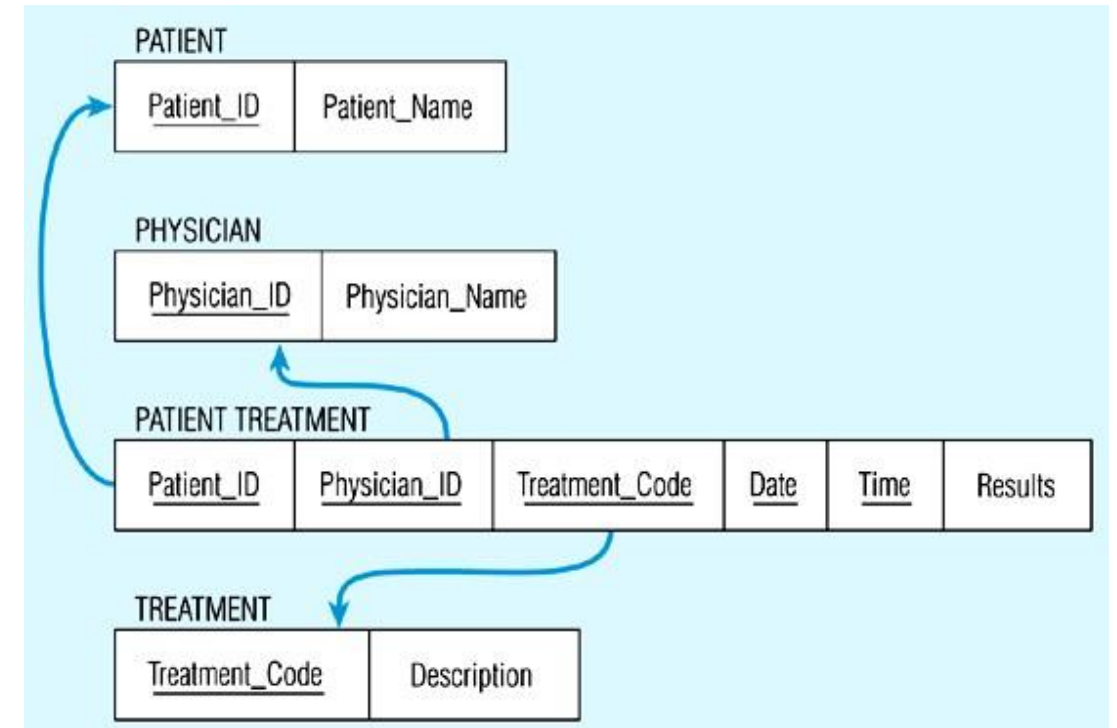
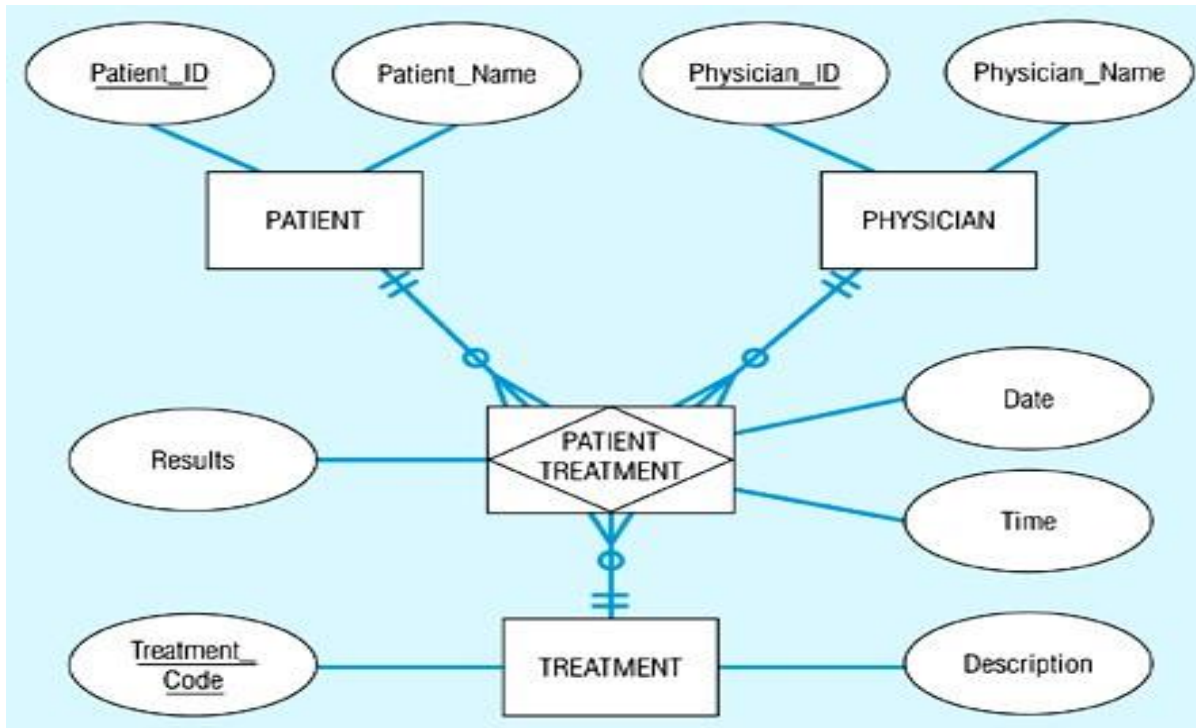
Cont..

❖ Example:

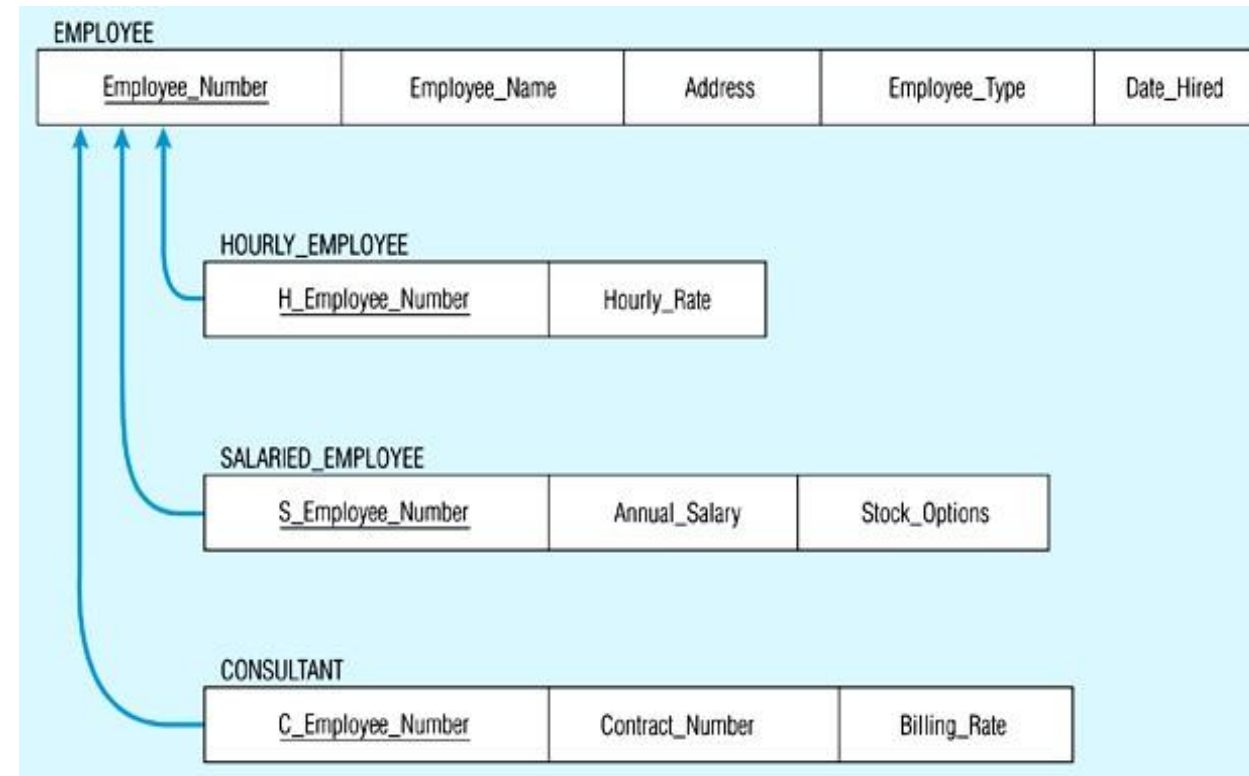
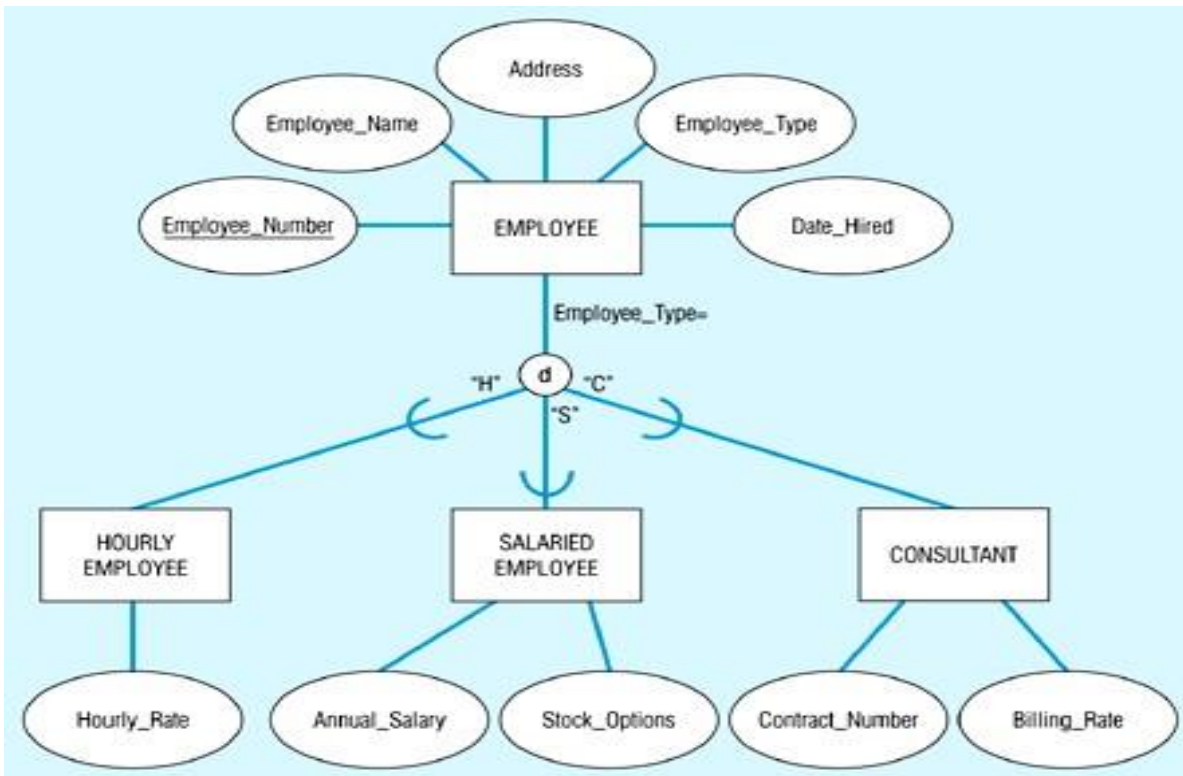


Mapping Ternary Relationships into Relations

- ❖ One relation for each entity and one for the associative entity.
- ❖ Associative entity has foreign keys to each entity in the relationship



Mapping Subtype/Supertype Relationships



Enhanced Entity Relationship(EER) Model

- ✓ EER is high level data model
- ✓ EER is the extension of ER model
- ✓ Why do we need it?
 - ❑ Some of the aspects of database such as inheritance among various entities can't be expressed using basic ER model. These aspects use Enhanced ER(EER) model.
- ✓ EER model concepts mainly concern
 - ❑ Sub class and Super class
 - ❑ Specialization and Generalization

Cont....

- ☐ Aggregation

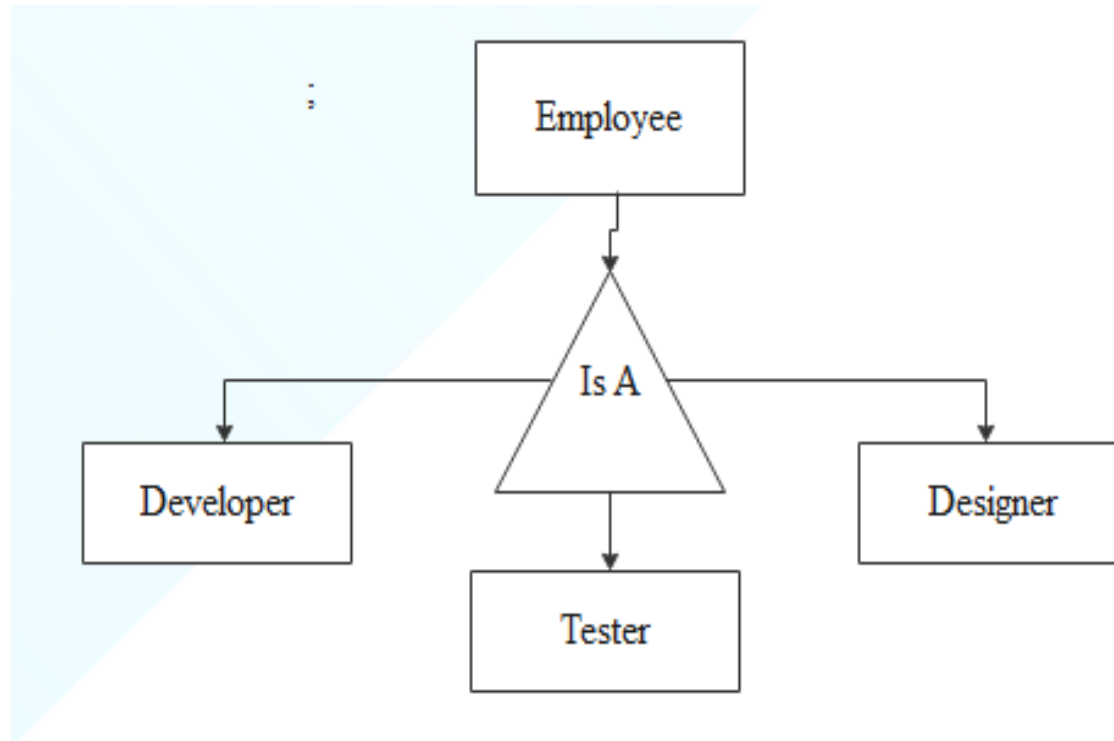
- ☐ Union or Category

- ✓ EER model includes all modelling concepts of ER model.
- ✓ From different concepts of EER model, generalization and specialization included.
- ✓ Specialization is based on what the role they play in an organization.

Example: An employee can be may be developer or tester or any other so that

This employee can be divided into subgroups.

Cont....

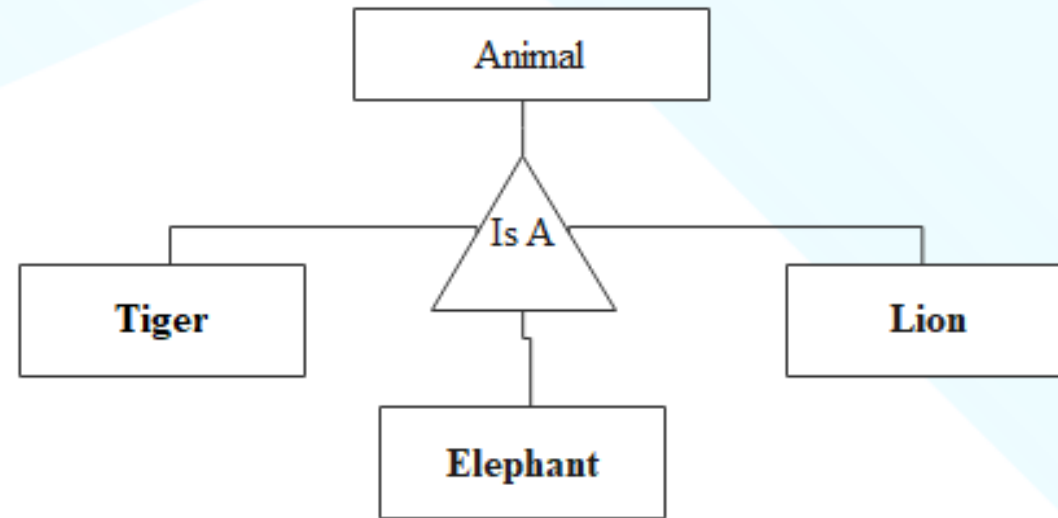


Cont....

From the above diagram

- ✓ **Specilazation** is the process that defines group entites in which divided into subgroups
- ✓ It is the top down approach
- ✓ And also in which one higher entity is broked down into lower entites.

Cont...



Cont...

From the above diagram

- ✓ **Generalization** is the process of generalizing the entities which contains the properties of all generalized entities
- ✓ In which two or more low level entities combined into single high level
- ✓ It is the bottom up approach
- ✓ It is the reverse process of specialization.

End of Chapter 3

Thank You!